

FREE SAMPLE CHAPTER

CHAPTER 5

GRADIENT DESCENT

and the Optimization Landscape

Convergence proofs · Adam & AdamW · the bias-correction
derivation · the “find the bug” divergence walkthrough

from Math for ML Engineers

FROM BACKPROP TO RLHF · THE DERIVATIONS BEHIND MODERN AI

Mikael Yemane · ISBN 979-8-9974520-0-1

github.com/mikaelyemane/ml-math-code

Gradient Descent and the Optimization Landscape

Prerequisites: Chapter 1: convexity, gradients, Hessians.

CHAPTER ROADMAP

You set `lr=1e-3` with Adam because it worked last time, and you treat convergence as something that either happens or doesn't.

By the end of this chapter, you will be able to:

- (1) prove the $O(1/T)$ convergence rate of gradient descent on smooth convex functions and sketch the Lyapunov argument behind the $O(1/T^2)$ rate for Nesterov's method;
- (2) derive the Adam update rule and explain the role of bias correction;
- (3) state KKT optimality conditions and the required constraint qualification;
- (4) identify the Polyak-Łojasiewicz condition and explain why it implies linear convergence on non-convex functions;
- (5) explain when $\beta_2 = 0.95$ should replace 0.999 for Transformer training, and how μP transfers learning rates across model width.

KEY IDEA · Hero Problem: Why Does Adam Dominate Transformers?

Train ResNet-50 on ImageNet with SGD + momentum: 90 epochs, days to weeks on the hardware of the era (the 2016 training took ~ 1 week on 8 GPUs; modern hardware completes the same run in hours), 76% top-1 accuracy. Train GPT-2 with SGD: it barely moves. Switch to Adam: loss drops immediately, and it becomes the default for every major language model.

What changed? ResNets have roughly isotropic loss curvature (all directions are similarly curved), so SGD’s uniform step size works. Transformers do not. The embedding gradients are orders of magnitude smaller than the attention gradients. A single global learning rate simultaneously under-steps one and over-steps the other.

Adam (Section 5.5) maintains a per-parameter running estimate of gradient magnitude and normalizes each update. The first-moment term m_t acts as momentum; the second-moment term v_t rescales each coordinate; and the bias-correction factors ensure the estimates are accurate from step 1. The derivation shows why this works: Adam implicitly adapts to the gradient magnitude of each dimension.

This chapter derives when and why each optimizer works, and when it does not.

RUNNING EXAMPLE · *Training the Exam Score Classifier*

Apply logistic regression to the pass/fail label on the exam dataset. Fit concretely: with unstandardized features ($x_1 \in [0, 10]$, $x_2 \in [0, 4]$, plus an intercept that has to reach ≈ -19.9 at the optimum) gradient descent with step $\eta = 0.01$ is nowhere near converged after 200 steps; it needs about 2.6×10^5 steps to close within 1% of the optimal loss, still 1.8% away even after 200,000. Adam at $\eta = 0.1$ closes to within 1% by step ≈ 800 : two-plus orders of magnitude fewer steps, because its per-coordinate scaling absorbs the scale mismatch between the intercept, x_1 , and x_2 that cripples a single constant step size. The $O(1/T)$ rate (Theorem 5.4, whose proof goes through for any $\eta \leq 1/L$ with $1/\eta$ in place of L) guarantees that after T steps at $\eta = 0.01$ (about a tenth of $1/L \approx 0.1$ here) the loss gap is at most $R^2/(2\eta T)$, an upper bound this run stays under throughout, though nowhere near tight.

Optimization is the mechanism by which every ML model learns, and every later chapter leans on the tools derived here. Gradient descent, Adam, Newton’s method — the choice determines how fast, or whether, training converges at all. This chapter derives why the usual hyperparameter instincts work.

5.1 Gradient Descent

INTUITION

You are standing on a hillside in thick fog, trying to reach the valley. All you can feel is the slope under your feet — the gradient. Step downhill, repeat. That’s the whole algorithm; everything else in this chapter is refinements on how big a step to take.

Definition 5.1 (Gradient Descent). Starting at $\boldsymbol{\theta}^{(0)}$, repeat:

$$\boxed{\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}^{(t)})}, \quad (5.1)$$

where $\eta > 0$ is the *learning rate* (step size).

Why does this decrease the loss? For an L -smooth $\mathcal{L}$ the *descent lemma* (a direct consequence of the integral form of Taylor’s theorem; see Appendix B, Calculus Reference) gives the inequality

$$\mathcal{L}(\boldsymbol{\theta} - \eta \mathbf{g}) \leq \mathcal{L}(\boldsymbol{\theta}) - \eta \|\mathbf{g}\|^2 + \frac{\eta^2 L}{2} \|\mathbf{g}\|^2, \quad (5.2)$$

where $\mathbf{g} = \nabla \mathcal{L}(\boldsymbol{\theta})$ and L is the smoothness constant (see below). For $\eta < 2/L$, the right-hand side is $\mathcal{L}(\boldsymbol{\theta}) - \eta(1 - \eta L/2) \|\mathbf{g}\|^2 < \mathcal{L}(\boldsymbol{\theta})$, and the choice $\eta = 1/L$ gives the canonical guarantee $\mathcal{L}(\boldsymbol{\theta}^{(t+1)}) \leq \mathcal{L}(\boldsymbol{\theta}^{(t)}) - \frac{1}{2L} \|\mathbf{g}\|^2$. So gradient descent with a small enough step always decreases the loss.

Choosing the learning rate. Too large: the quadratic approximation fails, loss can increase. Too small: progress is glacially slow. The right choice is $\eta \leq 1/L$ where L is the Lipschitz constant of the gradient.

Example 5.2 (Three Steps on the Exam Dataset). Watch Equation (5.1) actually run. Take the running example’s logistic loss (mean cross-entropy over the 500 students), standardize the two features, include an intercept, and start from $\boldsymbol{\theta}^{(0)} = \mathbf{0}$ with $\eta = 0.5$. At $\boldsymbol{\theta}^{(0)}$ every predicted probability is $\sigma(0) = \frac{1}{2}$, so the initial loss is exactly $\ln 2 \approx 0.6931$ — no simulation needed. The gradient there is also readable by hand: the intercept component is $\frac{1}{2} - \bar{y} \approx 0.27$ (only $\approx 23\%$ of students pass, and the model is predicting 50% for everyone), while the feature components are $\approx (-0.23, -0.19)$. The first step $\boldsymbol{\theta}^{(1)} = -\eta \nabla \mathcal{L}(\boldsymbol{\theta}^{(0)})$ therefore pushes the intercept *down* and both feature weights *up*; steepest descent spends most of its first move

correcting the grossest error (the base rate) while starting to credit study hours and GPA. The loss trace over three iterations:

$$0.6931 \rightarrow \approx 0.62 \rightarrow \approx 0.56 \rightarrow \approx 0.51,$$

each drop smaller than the last as $\|\nabla \mathcal{L}\|$ shrinks — at least the $-\eta(1 - \eta L/2) \|\mathbf{g}\|^2$ decrease that Equation (5.2) promises (the post- $\ln 2$ values vary in the second decimal with the sampling seed; the initial loss and the sign pattern of the first step do not).

— PRACTITIONER LENS —

WHEN THIS MATTERS. Any time you reach for the learning-rate hyperparameter in PyTorch or JAX. The Lipschitz bound L is rarely known a priori, so practitioners reach for a learning-rate finder (Smith 2017) that sweeps η exponentially and watches the loss; the optimal η is a few times below the value at which the loss starts to diverge ($\approx 2/L$), approximately corresponding to $1/L$ along the trajectory.

DEBUGGING SIGNAL. Plot the loss curve on log-y. A clean, monotonically decreasing line means $\eta < 1/L$ on average. A noisy zigzag with occasional spikes means η is too large for the worst-case curvature encountered; halve and continue.

PRODUCTION IMPACT. The cost of a wrong learning rate is paid in wall-clock time and discarded experiments. For a 7B-parameter model, a single training run is hundreds of thousands of dollars in compute; the LR sweep that finds the right η pays for itself many times over against a single diverged production run.

5.2 Convergence Analysis

Lemma 5.3 (Smoothness and Strong Convexity: the Four Facts Used Below). *Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be differentiable and L -smooth, meaning $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L \|\mathbf{x} - \mathbf{y}\|$ for all $\mathbf{x}, \mathbf{y}$. (a) Descent lemma. $f(\mathbf{y}) \leq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) + \frac{L}{2} \|\mathbf{y} - \mathbf{x}\|^2$. (b) Co-coercivity (Baillon–Haddad). If f is also convex, $(\nabla f(\mathbf{x}) - \nabla f(\mathbf{y}))^\top (\mathbf{x} - \mathbf{y}) \geq \frac{1}{L} \|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\|^2$. (c) Strong convexity implies PL. If f is μ -strongly convex with minimum value f^* , then $\frac{1}{2} \|\nabla f(\mathbf{x})\|^2 \geq \mu(f(\mathbf{x}) - f^*)$. (d) Optimal fixed step. For $f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x}$ with the eigenvalues of $\mathbf{A}$ in $[\mu, L]$ and $\kappa = L/\mu$, the step size minimizing the worst-case one-step contraction is $\eta = 2/(\mu + L)$, at which $\|\mathbf{x}^{(t+1)} - \mathbf{x}^*\| \leq \frac{\kappa-1}{\kappa+1} \|\mathbf{x}^{(t)} - \mathbf{x}^*\|$ and therefore $f(\mathbf{x}^{(t+1)}) - f^* \leq \left(\frac{\kappa-1}{\kappa+1}\right)^2 (f(\mathbf{x}^{(t)}) - f^*)$.*

Proof. (a) Write $\mathbf{d} = \mathbf{y} - \mathbf{x}$. The fundamental theorem of calculus along the segment gives $f(\mathbf{y}) - f(\mathbf{x}) = \int_0^1 \nabla f(\mathbf{x} + t\mathbf{d})^\top \mathbf{d} dt$, so, by Cauchy–Schwarz and L -smoothness,

$$f(\mathbf{y}) - f(\mathbf{x}) - \nabla f(\mathbf{x})^\top \mathbf{d} = \int_0^1 (\nabla f(\mathbf{x} + t\mathbf{d}) - \nabla f(\mathbf{x}))^\top \mathbf{d} dt \leq \int_0^1 Lt \|\mathbf{d}\|^2 dt = \frac{L}{2} \|\mathbf{d}\|^2.$$

(b) Fix $\mathbf{y}$ and set $\phi(\mathbf{z}) = f(\mathbf{z}) - \nabla f(\mathbf{y})^\top \mathbf{z}$, which is convex, L -smooth, and minimized at $\mathbf{z} = \mathbf{y}$ since $\nabla \phi(\mathbf{y}) = \mathbf{0}$. Applying (a) to ϕ at the point $\mathbf{z} - \frac{1}{L} \nabla \phi(\mathbf{z})$ gives $\phi(\mathbf{y}) \leq \phi(\mathbf{z}) - \frac{1}{2L} \|\nabla \phi(\mathbf{z})\|^2$; unfolding ϕ at $\mathbf{z} = \mathbf{x}$,

$$f(\mathbf{y}) \leq f(\mathbf{x}) + \nabla f(\mathbf{y})^\top (\mathbf{y} - \mathbf{x}) - \frac{1}{2L} \|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\|^2. \quad (5.3)$$

Adding Equation (5.3) to the same inequality with $\mathbf{x}$ and $\mathbf{y}$ exchanged cancels the function values and leaves (b).

(c) Strong convexity says $f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) + \frac{\mu}{2} \|\mathbf{y} - \mathbf{x}\|^2$ for every $\mathbf{y}$. Minimize both sides over $\mathbf{y}$: the left-hand side gives f^* , and the right-hand side is a quadratic minimized at $\mathbf{y} = \mathbf{x} - \frac{1}{\mu} \nabla f(\mathbf{x})$ with value $f(\mathbf{x}) - \frac{1}{2\mu} \|\nabla f(\mathbf{x})\|^2$. Rearranging gives (c).

(d) In the eigenbasis of $\mathbf{A}$ the step decouples as $\theta_i \mapsto (1 - \eta\lambda_i)\theta_i$, so the worst-case contraction is $\max\{|1 - \eta\mu|, |1 - \eta L|\}$, a piecewise-linear function of η minimized where its two branches meet, $1 - \eta\mu = \eta L - 1$, that is $\eta = 2/(\mu + L)$, where both equal $(L - \mu)/(L + \mu)$. Squaring carries the bound over to $f - f^* = \frac{1}{2} \sum_i \lambda_i \theta_i^2$. $\square$

Part (a) is the inequality quoted at Equation (5.2). Read at the pair $(\mathbf{x}^*, \mathbf{x})$ with $\nabla f(\mathbf{x}^*) = \mathbf{0}$, Equation (5.3) becomes $\nabla f(\mathbf{x})^\top (\mathbf{x} - \mathbf{x}^*) \geq f(\mathbf{x}) - f^* + \frac{1}{2L} \|\nabla f(\mathbf{x})\|^2$, the form Step 2 of the next proof uses. Part (c) supplies the Polyak–Łojasiewicz inequality with $\mu_{\text{PL}} = \mu$; the implication is strict, since PL requires neither convexity nor a unique minimizer, and it is PL rather than strong convexity that drives the linear rate of Theorems 5.5 and 5.8. Part (d) is the baseline $\rho_{\text{GD}} = (\kappa - 1)/(\kappa + 1)$ that Equation (5.23) measures momentum against.

Theorem 5.4 (GD Convergence for Convex Smooth Objectives). *Let $\mathcal{L}$ be convex and L -smooth. Assume a minimizer $\boldsymbol{\theta}^* \in \arg \min \mathcal{L}$ exists. Running GD with step size $\eta = 1/L$ for T steps, let $\bar{\boldsymbol{\theta}}^{(T)} = \frac{1}{T} \sum_{t=0}^{T-1} \boldsymbol{\theta}^{(t)}$ be the average iterate and $R := \|\boldsymbol{\theta}^{(0)} - \boldsymbol{\theta}^*\|$. The average iterate achieves an $O(1/T)$ rate:*

$$\mathcal{L}(\bar{\boldsymbol{\theta}}^{(T)}) - \mathcal{L}^* \leq \frac{LR^2}{2T} = O\left(\frac{1}{T}\right). \quad (5.4)$$

From Equation (5.8), $\frac{1}{T} \sum_{t=0}^{T-1} (\mathcal{L}(\boldsymbol{\theta}^{(t)}) - \mathcal{L}^*) \leq \frac{LR^2}{2T}$. Since $\{\mathcal{L}(\boldsymbol{\theta}^{(t)})\}$ is monotone non-increasing (by Equation (5.5)), $\mathcal{L}(\boldsymbol{\theta}^{(T)}) \leq \mathcal{L}(\boldsymbol{\theta}^{(T-1)}) = \min_{t=0, \dots, T-1} \mathcal{L}(\boldsymbol{\theta}^{(t)}) \leq$

$\frac{1}{T} \sum_{t=0}^{T-1} \mathcal{L}(\boldsymbol{\theta}^{(t)})$, so $\mathcal{L}(\boldsymbol{\theta}^{(T)}) - \mathcal{L}^* \leq \frac{1}{T} \sum_{t=0}^{T-1} (\mathcal{L}(\boldsymbol{\theta}^{(t)}) - \mathcal{L}^*)$, and the last iterate achieves the same $O(1/T)$ bound. Under additional strong convexity the last iterate achieves linear convergence; see Theorem 5.5.

Proof. Step 1 (Descent lemma). Since $\mathcal{L}$ is L -smooth, substituting the GD step into the smoothness inequality gives:

$$\mathcal{L}(\boldsymbol{\theta}^{(t+1)}) \leq \mathcal{L}(\boldsymbol{\theta}^{(t)}) - \frac{1}{2L} \left\| \nabla \mathcal{L}(\boldsymbol{\theta}^{(t)}) \right\|^2. \quad (5.5)$$

In particular $\{\mathcal{L}(\boldsymbol{\theta}^{(t)})\}$ is monotone non-increasing.

Step 2 (Distance potential). Expand the squared distance to $\boldsymbol{\theta}^*$ after one step:

$$\begin{aligned} \left\| \boldsymbol{\theta}^{(t+1)} - \boldsymbol{\theta}^* \right\|^2 &= \left\| \boldsymbol{\theta}^{(t)} - \frac{1}{L} \nabla \mathcal{L}^{(t)} - \boldsymbol{\theta}^* \right\|^2 \\ &= \left\| \boldsymbol{\theta}^{(t)} - \boldsymbol{\theta}^* \right\|^2 - \frac{2}{L} \langle \nabla \mathcal{L}^{(t)}, \boldsymbol{\theta}^{(t)} - \boldsymbol{\theta}^* \rangle + \frac{1}{L^2} \left\| \nabla \mathcal{L}^{(t)} \right\|^2. \end{aligned} \quad (5.6)$$

For a convex L -smooth function, the co-coercivity bound (requiring both convexity and L -smoothness; Baillon–Haddad) gives $\langle \nabla \mathcal{L}^{(t)}, \boldsymbol{\theta}^{(t)} - \boldsymbol{\theta}^* \rangle \geq \mathcal{L}^{(t)} - \mathcal{L}^* + \frac{1}{2L} \left\| \nabla \mathcal{L}^{(t)} \right\|^2$. Substituting into the distance expansion and canceling the gradient-norm terms:

$$\left\| \boldsymbol{\theta}^{(t+1)} - \boldsymbol{\theta}^* \right\|^2 \leq \left\| \boldsymbol{\theta}^{(t)} - \boldsymbol{\theta}^* \right\|^2 - \frac{2}{L} (\mathcal{L}^{(t)} - \mathcal{L}^*). \quad (5.7)$$

Step 3 (Telescope and average). Rearrange Equation (5.7) and sum $t = 0, \dots, T-1$:

$$\begin{aligned} \frac{2}{L} \sum_{t=0}^{T-1} (\mathcal{L}^{(t)} - \mathcal{L}^*) &\leq R^2 - \left\| \boldsymbol{\theta}^{(T)} - \boldsymbol{\theta}^* \right\|^2 \\ &\leq R^2. \end{aligned} \quad (5.8)$$

Divide by $2T/L$ and apply convexity of $\mathcal{L}$ ($\mathcal{L}(\bar{\boldsymbol{\theta}}^{(T)}) \leq \frac{1}{T} \sum_t \mathcal{L}(\boldsymbol{\theta}^{(t)})$):

$$\mathcal{L}(\bar{\boldsymbol{\theta}}^{(T)}) - \mathcal{L}^* \leq \frac{LR^2}{2T} = O\left(\frac{1}{T}\right). \quad \square \quad (5.9)$$

□

Theorem 5.5 (GD Convergence for Strongly Convex Smooth Objectives). *If $\mathcal{L}$ is μ -strongly convex and L -smooth (with condition number $\kappa = L/\mu$), then a minimizer $\boldsymbol{\theta}^*$ exists and is unique, and GD with $\eta = 1/L$ converges linearly to the last iterate:*

$$\mathcal{L}(\boldsymbol{\theta}^{(T)}) - \mathcal{L}^* \leq \left(1 - \frac{1}{\kappa}\right)^T [\mathcal{L}(\boldsymbol{\theta}^{(0)}) - \mathcal{L}^*]. \quad (5.10)$$

Proof sketch. Distance-based route (yields a weaker bound). Expanding $\|\boldsymbol{\theta}^{(t+1)} - \boldsymbol{\theta}^*\|^2 = \|\boldsymbol{\theta}^{(t)} - \eta \nabla \mathcal{L} - \boldsymbol{\theta}^*\|^2$ gives three terms; the cross term $-2\eta \langle \nabla \mathcal{L}, \boldsymbol{\theta}^{(t)} - \boldsymbol{\theta}^* \rangle$ and the squared-gradient term $\eta^2 \|\nabla \mathcal{L}\|^2$. (i) Apply co-coercivity (requires L -smoothness *and* convexity; Baillon–Haddad theorem) to bound $\|\nabla \mathcal{L}\|^2 \leq L \langle \nabla \mathcal{L}, \boldsymbol{\theta}^{(t)} - \boldsymbol{\theta}^* \rangle$ (since $\nabla \mathcal{L}(\boldsymbol{\theta}^*) = 0$ at the minimizer), which with $\eta = 1/L$ makes the $\eta^2 \|\nabla \mathcal{L}\|^2$ term cancel half the cross term. (ii) Apply the symmetrized strong-convexity inequality $\langle \nabla \mathcal{L}(\boldsymbol{\theta}^{(t)}), \boldsymbol{\theta}^{(t)} - \boldsymbol{\theta}^* \rangle \geq \mu \|\boldsymbol{\theta}^{(t)} - \boldsymbol{\theta}^*\|^2$ (obtained by summing the two one-sided strong-convexity inequalities and using $\nabla \mathcal{L}(\boldsymbol{\theta}^*) = 0$) to bound the remaining $-\frac{1}{L} \langle \nabla \mathcal{L}, \boldsymbol{\theta}^{(t)} - \boldsymbol{\theta}^* \rangle \leq -\frac{\mu}{L} \|\boldsymbol{\theta}^{(t)} - \boldsymbol{\theta}^*\|^2$. Combining gives $\|\boldsymbol{\theta}^{(t+1)} - \boldsymbol{\theta}^*\|^2 \leq (1 - 1/\kappa) \|\boldsymbol{\theta}^{(t)} - \boldsymbol{\theta}^*\|^2$, so by induction $\|\boldsymbol{\theta}^{(T)} - \boldsymbol{\theta}^*\|^2 \leq (1 - 1/\kappa)^T \|\boldsymbol{\theta}^{(0)} - \boldsymbol{\theta}^*\|^2$. Converting via L -smoothness ($\mathcal{L}(\boldsymbol{\theta}) - \mathcal{L}^* \leq \frac{L}{2} \|\boldsymbol{\theta} - \boldsymbol{\theta}^*\|^2$) gives $\mathcal{L}(\boldsymbol{\theta}^{(T)}) - \mathcal{L}^* \leq \frac{L}{2} (1 - 1/\kappa)^T \|\boldsymbol{\theta}^{(0)} - \boldsymbol{\theta}^*\|^2$, which involves $\|\boldsymbol{\theta}^{(0)} - \boldsymbol{\theta}^*\|^2$, not $\mathcal{L}(\boldsymbol{\theta}^{(0)}) - \mathcal{L}^*$.

Function-value route (establishes the stated bound (5.10) directly). The descent lemma gives $\mathcal{L}(\boldsymbol{\theta}^{(t+1)}) \leq \mathcal{L}(\boldsymbol{\theta}^{(t)}) - \frac{1}{2L} \|\nabla \mathcal{L}(\boldsymbol{\theta}^{(t)})\|^2$. Strong convexity implies the Polyak–Łojasiewicz condition $\|\nabla \mathcal{L}\|^2 \geq 2\mu(\mathcal{L} - \mathcal{L}^*)$, so

$$\mathcal{L}(\boldsymbol{\theta}^{(t+1)}) - \mathcal{L}^* \leq \mathcal{L}(\boldsymbol{\theta}^{(t)}) - \mathcal{L}^* - \frac{\mu}{L} (\mathcal{L}(\boldsymbol{\theta}^{(t)}) - \mathcal{L}^*) = \left(1 - \frac{1}{\kappa}\right) (\mathcal{L}(\boldsymbol{\theta}^{(t)}) - \mathcal{L}^*).$$

Iterating T times yields the stated bound (5.10) directly in function values. $\square$

Remark 5.6. Strong convexity implies coercivity ($\mathcal{L}(\boldsymbol{\theta}) \rightarrow \infty$ as $\|\boldsymbol{\theta}\| \rightarrow \infty$), so the minimum is attained; coercivity need not be assumed separately.

INTUITION

Linear convergence means the error shrinks by a constant factor every step: $\delta_{t+1} \leq (1 - 1/\kappa) \delta_t$. Small condition number, fast convergence. Large κ and the loss landscape looks like a narrow ravine, and gradient descent zigzags across it instead of running down it. Preconditioning (second-order methods, batch normalization) exists to fix this.

Definition 5.7 (Polyak–Łojasiewicz (PL) Condition). A differentiable function f with finite infimum $f^* := \inf_{\mathbf{x}} f(\mathbf{x}) > -\infty$ satisfies the PL condition with constant $\mu_{\text{PL}} > 0$ if for all $\mathbf{x}$:

$$\frac{1}{2} \|\nabla f(\mathbf{x})\|^2 \geq \mu_{\text{PL}} (f(\mathbf{x}) - f^*).$$

(Notation: μ_{PL} is distinct from the strong-convexity constant μ used above.)

Theorem 5.8 (Linear Convergence under PL). *If f is L -smooth and satisfies the PL condition globally (for all $\mathbf{x} \in \mathbb{R}^d$) with constant μ_{PL} , then gradient descent with step size $\eta = 1/L$ converges linearly: $f(\mathbf{x}^{(t)}) - f^* \leq (1 - \mu_{\text{PL}}/L)^t (f(\mathbf{x}^{(0)}) - f^*)$.*

Remark 5.9. The PL condition is weaker than strong convexity (it does not require a unique minimizer) and is satisfied (in the PL* form, with $f^* = 0$) by sufficiently wide overparameterized neural networks in a ball around initialization large enough to contain a global minimizer [Liu et al., 2022]. This provides theoretical justification for the empirical observation that gradient descent reliably minimizes training loss in overparameterized models. A local PL condition (holding only in a neighborhood of a minimizer) implies only local convergence to that minimizer; global PL (holding for all $\mathbf{x} \in \mathbb{R}^d$, as in Theorem 5.8) is needed for global convergence guarantees.

PRACTITIONER LENS

WHEN THIS MATTERS. When a paper claims a convergence rate, look for the assumed conditions. For SGD on a general non-convex deep network, the Ghadimi–Lan (2013) $O(1/\sqrt{T})$ bound on $\min_t \mathbb{E}[\|\nabla f\|^2]$ applies under L -smoothness and bounded variance; the PL condition is the closest practical regularity that overparameterized networks actually satisfy near the minimum.

DEBUGGING SIGNAL. Training loss plateauing at a value strictly greater than $\mathcal{L}^*$ on a fully overparameterized network is usually evidence of optimizer issues (LR, gradient clipping, mixed-precision underflow) rather than a fundamental capacity problem: the PL theorem says a correctly tuned GD-like optimizer will drive training loss to zero.

PRODUCTION IMPACT. The “interpolation regime” applies to smaller overparameterized settings (more parameters than training samples), where PL/NTK theory guarantees that a well-tuned optimizer drives training loss to zero. Modern frontier LLMs operate in the opposite regime: Llama-3-70B trains on $\sim 15\text{T}$ tokens with only 70B parameters (a $200\times$ data-to-parameter ratio), so training loss never gets close to zero; instead it plateaus at the language-modeling entropy floor ($\approx 1.5\text{--}2.5$ nats). The rate-limiting step in LLM pretraining is compute budget and data quality, not interpolation-regime convergence theory.

5.3 Stochastic Gradient Descent

For n data points, computing $\nabla \mathcal{L} = \frac{1}{n} \sum_i \nabla \mathcal{L}_i$ is $O(n)$ per step — too slow for large datasets. SGD approximates the gradient using a *minibatch*:

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta_t \nabla_{\boldsymbol{\theta}} \mathcal{L}_{\mathcal{B}}(\boldsymbol{\theta}^{(t)}), \quad \mathcal{L}_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \mathcal{L}_i, \quad (5.11)$$

where $\mathcal{B}$ is a random minibatch.

Noise analysis. The stochastic gradient $\mathbf{g}_t = \nabla \mathcal{L}_{\mathcal{B}}$ satisfies:

$$\mathbb{E}[\mathbf{g}_t] = \nabla \mathcal{L}(\boldsymbol{\theta}^{(t)}) \quad (\text{unbiased}) \quad (5.12)$$

$$\text{Var}[\mathbf{g}_t] \approx \frac{\sigma^2}{|\mathcal{B}|} \quad (\text{variance shrinks with batch size}). \quad (5.13)$$

Remark 5.10 (Minibatch Variance, Linear Scaling, and the Critical Batch Size). Write $\sigma^2 := \frac{1}{n} \sum_{i=1}^n \|\nabla \mathcal{L}_i(\boldsymbol{\theta}) - \nabla \mathcal{L}(\boldsymbol{\theta})\|^2$ for the per-sample gradient variance and $B = |\mathcal{B}|$. Drawing the B indices i.i.d. uniformly makes $\nabla \mathcal{L}_{\mathcal{B}}$ an average of B i.i.d. unbiased terms, so its variance is that of one term divided by B :

$$\mathbb{E} \|\nabla \mathcal{L}_{\mathcal{B}}(\boldsymbol{\theta}) - \nabla \mathcal{L}(\boldsymbol{\theta})\|^2 = \frac{\sigma^2}{B} \quad (5.14)$$

(sampling without replacement multiplies this by the finite-population factor $(n - B)/(n - 1)$).

Linear scaling. Compare k steps at (B, η) against one step at $(kB, k\eta)$, holding $\nabla \mathcal{L}$ fixed across the k small steps. Both move $k\eta \nabla \mathcal{L}$ in expectation, and the accumulated noise variance is $k \cdot \eta^2 \sigma^2 / B$ for the first and $(k\eta)^2 \sigma^2 / (kB) = k\eta^2 \sigma^2 / B$ for the second: equal drift, equal noise, so multiplying B by k and η by k leaves the noise-to-signal ratio per unit of progress unchanged. The premise is that the gradient does not move over the k small steps, which by Theorem 5.3(a) needs the quadratic term $\frac{L\eta^2}{2} \|\mathbf{g}\|^2$ to stay negligible against the linear term $\eta \|\mathbf{g}\|^2$, i.e. $\eta \ll 2/L$. The rule therefore holds only until $k\eta$ approaches $1/L$, which is why linear scaling fails at very large batch.

Critical batch size. Take expectations in Theorem 5.3(a) along the step $-\eta \nabla \mathcal{L}_{\mathcal{B}}$, write $G = \|\nabla \mathcal{L}\|^2$, and use Equation (5.14) to get $\mathbb{E}[\mathcal{L}(\boldsymbol{\theta}^{(t+1)})] - \mathcal{L}(\boldsymbol{\theta}^{(t)}) \leq -\eta G + \frac{L\eta^2}{2}(G + \sigma^2/B)$. Optimizing over η gives $\eta^* = G/[L(G + \sigma^2/B)]$ and a per-step decrease of

$$\frac{G}{2L} \cdot \frac{B}{B + B_{\text{crit}}}, \quad B_{\text{crit}} := \frac{\sigma^2}{\|\nabla \mathcal{L}\|^2}. \quad (5.15)$$

Below B_{crit} the decrease is nearly proportional to B , so a batch twice as expensive buys close to twice the progress; above it the factor saturates at 1, variance reduction

stops buying speed, and the cost per step keeps climbing. That saturation point is what the scaling literature means by the *critical batch size* (equivalently the gradient noise scale); it grows during training as $\|\nabla \mathcal{L}\|$ falls.

Theorem 5.11 (SGD Convergence Rate). *For convex $\mathcal{L}$ with $\mathbb{E}[\|\mathbf{g}_t\|^2] \leq G^2$ (bounded second moment of stochastic gradients) and initial distance $R_0 = \|\boldsymbol{\theta}^{(0)} - \boldsymbol{\theta}^*\|$, with constant step $\eta_t = R_0/(G\sqrt{T})$, after T steps:*

$$\mathbb{E}[\mathcal{L}(\bar{\boldsymbol{\theta}}^{(T)})] - \mathcal{L}^* = O\left(\frac{GR_0}{\sqrt{T}}\right), \quad (5.16)$$

where $\bar{\boldsymbol{\theta}}^{(T)} = \frac{1}{T} \sum_{t=0}^{T-1} \boldsymbol{\theta}^{(t)}$ is the averaged iterate. The decaying schedule $\eta_t = c/\sqrt{t}$ for $t = 1, \dots, T$ (undefined at $t = 0$; use $\eta_0 = \eta_1$) achieves $O(\log T/\sqrt{T})$ for the η_t -weighted average $\bar{\boldsymbol{\theta}}^{(T)} = (\sum_t \eta_t \boldsymbol{\theta}^{(t)})/(\sum_t \eta_t)$; for constant step this equals $\bar{\boldsymbol{\theta}}^{(T)}$, but for non-constant steps the bound applies to the weighted, not the uniform, average.

Proof sketch. Let $R_t = \|\boldsymbol{\theta}^{(t)} - \boldsymbol{\theta}^*\|$. Expanding:

$$R_{t+1}^2 = \|\boldsymbol{\theta}^{(t)} - \eta_t \mathbf{g}_t - \boldsymbol{\theta}^*\|^2 = R_t^2 - 2\eta_t \mathbf{g}_t^\top (\boldsymbol{\theta}^{(t)} - \boldsymbol{\theta}^*) + \eta_t^2 \|\mathbf{g}_t\|^2.$$

Taking conditional expectation given $\boldsymbol{\theta}^{(t)}$ and using unbiasedness $\mathbb{E}[\mathbf{g}_t | \boldsymbol{\theta}^{(t)}] = \nabla \mathcal{L}(\boldsymbol{\theta}^{(t)})$, convexity gives $\mathbb{E}[\mathbf{g}_t^\top (\boldsymbol{\theta}^{(t)} - \boldsymbol{\theta}^*) | \boldsymbol{\theta}^{(t)}] = \nabla \mathcal{L}(\boldsymbol{\theta}^{(t)})^\top (\boldsymbol{\theta}^{(t)} - \boldsymbol{\theta}^*) \geq \mathcal{L}(\boldsymbol{\theta}^{(t)}) - \mathcal{L}^*$. Taking full expectation (using $\mathbb{E}[\|\mathbf{g}_t\|^2] \leq G^2$):

$$2\eta_t \mathbb{E}[\mathcal{L}(\boldsymbol{\theta}^{(t)}) - \mathcal{L}^*] \leq \mathbb{E}[R_t^2] - \mathbb{E}[R_{t+1}^2] + \eta_t^2 G^2.$$

Summing over $t = 0, \dots, T-1$, the R_t^2 terms telescope. Dividing by $2 \sum_t \eta_t$ and applying Jensen's inequality to $\tilde{\boldsymbol{\theta}}^{(T)} = (\sum_t \eta_t \boldsymbol{\theta}^{(t)})/(\sum_t \eta_t)$ (where $\tilde{\boldsymbol{\theta}}^{(T)}$ is the step-size-weighted average; Jensen requires this for non-constant step sizes) gives

$$\mathbb{E}[\mathcal{L}(\tilde{\boldsymbol{\theta}}^{(T)})] - \mathcal{L}^* \leq \frac{R_0^2 + G^2 \sum_t \eta_t^2}{2 \sum_t \eta_t}.$$

Setting $\eta_t = R_0/(G\sqrt{T})$ (constant step) yields the $O(GR_0/\sqrt{T})$ bound. The decaying schedule $\eta_t = c/\sqrt{t}$ (for $t \geq 1$) achieves rate $O(\log T/\sqrt{T})$ with $\sum_t \eta_t^2 = O(\log T)$, $\sum_t \eta_t = O(\sqrt{T})$. The $\log T$ factor comes from $\sum_t \eta_t^2 = c^2 \sum_t 1/t = O(\log T)$ (harmonic series); $\sum_t \eta_t = c \sum_t 1/\sqrt{t} = O(\sqrt{T})$. $\square$

KEY IDEA

GD achieves $O(1/T)$ for both the average and last iterate on smooth convex objectives (Theorem 5.4); SGD achieves $O(1/\sqrt{T})$. GD is faster *per step*, but each GD step costs $O(n)$ (full dataset) while SGD costs $O(|\mathcal{B}|)$, so at fixed wall-clock time SGD completes far more parameter updates. Gradient noise also acts as a regularizer that helps escape sharp minima. For large datasets, SGD (or Adam) is always preferred; GD’s faster asymptotic rate is a per-iteration, not per-second, statement.

PRACTITIONER LENS

WHEN THIS MATTERS. Every time you pick a batch size. The SGD noise variance scales as $\sigma^2/|\mathcal{B}|$, so doubling the batch size halves the gradient noise, but also halves the regularizing effect. The “linear scaling rule” (Goyal et al. 2017) preserves per-step update dynamics by scaling η linearly with $|\mathcal{B}|$ up to a critical batch size.

DEBUGGING SIGNAL. Train loss is too noisy: batch is too small, or LR is too high relative to noise. Train loss decreases smoothly but val loss plateaus: batch is too large; SGD noise is doing useful regularization that you have removed. Look at the variance of the gradient norm within a batch.

PRODUCTION IMPACT. For LLM pretraining at the 1M+ token-batch scale, the critical batch size is empirically $\approx 0.5\text{--}2\text{M}$ tokens (McCandlish et al. 2018). Beyond it, larger batches stop cutting the number of optimizer steps (the gradient is already approximately the full-batch gradient), so additional GPUs buy little wall-clock speedup while consuming proportionally more tokens per unit of progress.

5.4 Momentum and Nesterov Acceleration

5.4.1 Heavy Ball Momentum

The classical heavy-ball method [Polyak, 1964] introduced momentum as a way to accelerate gradient descent by adding inertia to the iterates.

INTUITION

Gradient descent in a narrow valley zigzags — the gradient keeps flipping direction across the valley while barely progressing along it. Momentum accumulates velocity along the gradient direction, which damps the zigzag and

gets you somewhere.

$$\mathbf{v}^{(t+1)} = \beta \mathbf{v}^{(t)} + (1 - \beta) \nabla \mathcal{L}(\boldsymbol{\theta}^{(t)}) \quad (5.17)$$

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \mathbf{v}^{(t+1)}. \quad (5.18)$$

Here $\beta \in [0, 1)$ is the momentum coefficient. The velocity $\mathbf{v}^{(t)}$ is an exponential moving average of past gradients:

$$\mathbf{v}^{(t)} = (1 - \beta) \sum_{k=0}^{t-1} \beta^{t-1-k} \nabla \mathcal{L}(\boldsymbol{\theta}^{(k)}). \quad (5.19)$$

COMMON MISTAKE

Implementation convention. PyTorch `SGD(momentum= $\beta$ )` uses the convention $\mathbf{v} \leftarrow \beta \mathbf{v} + \nabla \mathcal{L}$ *without* the $(1 - \beta)$ factor, which gives $\|\mathbf{v}\|_\infty \sim \|\nabla\|_\infty / (1 - \beta)$ in steady state. The form above with $(1 - \beta)$ keeps $\mathbf{v}$ on the same scale as $\nabla \mathcal{L}$ and matches the EMA interpretation. Adam uses the $(1 - \beta)$ form for both moments. When reading code, always check which convention is in use.

DERIVATION · Why Momentum Accelerates: the Quadratic Analysis

Take $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2} \boldsymbol{\theta}^\top \mathbf{A} \boldsymbol{\theta}$ with $\mathbf{A} \succ \mathbf{0}$, eigenvalues in $[m, L]$, and condition number $\kappa = L/m$. Write heavy ball in its classical two-term form (equivalent to Equations (5.17) and (5.18) up to a reparametrization of η):

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \alpha \nabla \mathcal{L}(\boldsymbol{\theta}^{(t)}) + \mu(\boldsymbol{\theta}^{(t)} - \boldsymbol{\theta}^{(t-1)}). \quad (5.20)$$

In the eigenbasis of $\mathbf{A}$ the update decouples: along an eigenvector with eigenvalue λ , the scalar coordinate θ_t obeys the linear recursion

$$\theta_{t+1} = (1 + \mu - \alpha\lambda) \theta_t - \mu \theta_{t-1}, \quad \begin{pmatrix} \theta_{t+1} \\ \theta_t \end{pmatrix} = \underbrace{\begin{pmatrix} 1 + \mu - \alpha\lambda & -\mu \\ 1 & 0 \end{pmatrix}}_{\mathbf{T}_\lambda} \begin{pmatrix} \theta_t \\ \theta_{t-1} \end{pmatrix}, \quad (5.21)$$

so the asymptotic per-step contraction is the spectral radius $\rho(\mathbf{T}_\lambda)$, maximized over $\lambda \in [m, L]$. The characteristic polynomial of the companion matrix $\mathbf{T}_\lambda$ is $r^2 - (1 + \mu - \alpha\lambda)r + \mu = 0$. When its discriminant $(1 + \mu - \alpha\lambda)^2 - 4\mu$ is ≤ 0 , the roots are a complex conjugate pair whose product is μ , so *both* have

modulus exactly $\sqrt{\mu}$ — independent of λ . Choosing

$$\alpha = \frac{4}{(\sqrt{L} + \sqrt{m})^2}, \quad \boxed{\mu = \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^2} \quad (5.22)$$

makes the discriminant ≤ 0 across the whole spectrum, with equality exactly at the endpoints: at $\lambda = m$ one checks in one line that $1 + \mu - \alpha m = 2\sqrt{\mu}$ (and at $\lambda = L$, $1 + \mu - \alpha L = -2\sqrt{\mu}$), so every eigendirection contracts at

$$\rho_{\text{HB}} = \sqrt{\mu} = \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \quad \text{vs.} \quad \rho_{\text{GD}} = \frac{\kappa - 1}{\kappa + 1} \quad (5.23)$$

for gradient descent with its optimal step $\eta = 2/(m + L)$ (Section 5.2). The dependence improves from κ to $\sqrt{\kappa}$: momentum is worth a square root of the condition number.

Example 5.12 ($\kappa = 100$: a Square Root You Can Feel). For $\kappa = 100$, Equation (5.23) gives $\rho_{\text{GD}} = 99/101 \approx 0.980$ and $\rho_{\text{HB}} = 9/11 \approx 0.818$, with optimal momentum $\mu = (9/11)^2 \approx 0.669$. Iterations needed per decimal digit of accuracy are $\ln 10/(-\ln \rho)$:

$$\text{GD: } \frac{\ln 10}{-\ln 0.980} \approx 115 \text{ iterations/digit}, \quad \text{HB: } \frac{\ln 10}{-\ln 0.818} \approx 11.5 \text{ iterations/digit},$$

a $10\times$ speedup ($\ln 0.980/\ln 0.818 \approx 1/10.0$), and the gap widens as $\sqrt{\kappa}$ for worse conditioning. A direct simulation on $\mathbf{A} = \text{diag}(1, 100)$ confirms it: over 200 iterations the measured per-step contraction is 0.980 for GD and 0.841 for heavy ball (slightly above the asymptotic 0.818 because the double root at the spectrum's edges contributes a decaying polynomial transient).

5.4.2 Nesterov Accelerated Gradient

Nesterov's insight: evaluate the gradient *after* taking the momentum step (a “lookahead” gradient). The $O(1/T^2)$ rate requires a *time-varying* momentum

schedule:

$$\boldsymbol{\theta}^{(1)} = \boldsymbol{\theta}^{(0)} - \frac{1}{L} \nabla \mathcal{L}(\boldsymbol{\theta}^{(0)}) \quad (5.24)$$

$$t_{k+1} = \frac{1}{2}(1 + \sqrt{1 + 4t_k^2}), \quad t_1 = 1 \quad (5.25)$$

$$\beta_k = (t_k - 1)/t_{k+1} \quad (5.26)$$

$$\tilde{\boldsymbol{\theta}}^{(k)} = \boldsymbol{\theta}^{(k)} + \beta_k(\boldsymbol{\theta}^{(k)} - \boldsymbol{\theta}^{(k-1)}) \quad (5.27)$$

$$\boldsymbol{\theta}^{(k+1)} = \tilde{\boldsymbol{\theta}}^{(k)} - \frac{1}{L} \nabla \mathcal{L}(\tilde{\boldsymbol{\theta}}^{(k)}). \quad (5.28)$$

A fixed β gives the *Heavy Ball* method, which has no general $O(1/T^2)$ guarantee for convex objectives.

Theorem 5.13 (Nesterov Convergence Rate). *Let $\mathcal{L}$ be convex and L -smooth. Nesterov's accelerated gradient method with step size $\eta = 1/L$ and the t_k schedule in Equation (5.25) satisfies*

$$\mathcal{L}(\boldsymbol{\theta}^{(T)}) - \mathcal{L}^* \leq \frac{2L \|\boldsymbol{\theta}^{(0)} - \boldsymbol{\theta}^*\|^2}{(T+1)^2} = O(1/T^2). \quad (5.29)$$

This $O(1/T^2)$ rate is optimal for first-order methods on smooth convex functions [Nesterov, 1983].

Proof sketch. See the Lyapunov-function argument below. □

Remark 5.14. The $O(1/T^2)$ rate is *optimal* for first-order methods on smooth convex problems (no algorithm using only gradient evaluations can guarantee a faster rate in the worst case). This is why Nesterov's method is preferred for smooth convex problems (FISTA, some SGD variants) and explains the theoretical motivation for momentum in general; Adam and AdamW use a simpler heavy-ball momentum, not the full Nesterov schedule.

INTUITION

The t_k sequence grows as $t_k \approx k/2$ asymptotically. The exact identity $t_{k+1}(t_{k+1} - 1) = t_k^2$ (verified by substituting the recurrence) gives $t_{k+1}^2 = t_k^2 + t_{k+1}$, so $t_{k+1} - t_k \rightarrow \frac{1}{2}$, meaning $t_k = \Theta(k)$ with implicit constant $1/2$ (i.e. $t_k \sim k/2$). Hence $\beta_k = (t_k - 1)/t_{k+1} \rightarrow 1$. The increasing momentum fraction gives the accumulated direction more weight as iteration count grows, improving the rate from gradient descent's $O(1/T)$ to $O(1/T^2)$. The proof uses a Lyapunov function $\phi_k = t_k^2[\mathcal{L}(\boldsymbol{\theta}^{(k)}) - \mathcal{L}^*] + \frac{L}{2} \|t_k \boldsymbol{\theta}^{(k)} - (t_k - 1) \boldsymbol{\theta}^{(k-1)} - \boldsymbol{\theta}^*\|^2$

that telescopes to $O(1/T^2)$ with this specific β_k schedule.

Why ϕ_k telescopes. Each step the descent lemma plus the identity $t_{k+1}(t_{k+1} - 1) = t_k^2$ give $\phi_k - \phi_{k-1} \leq 0$ (Nesterov 2018, Theorem 2.2.2). A one-step descent argument with $\mathbf{y}^{(1)} = \boldsymbol{\theta}^{(0)}$, $t_1 = 1$ gives the base case $\phi_1 \leq \frac{L}{2} \|\boldsymbol{\theta}^{(0)} - \boldsymbol{\theta}^*\|^2$: using the descent lemma plus convexity and expanding $\|\boldsymbol{\theta}^{(1)} - \boldsymbol{\theta}^*\|^2 = \|\boldsymbol{\theta}^{(0)} - \frac{1}{L} \nabla \mathcal{L}(\boldsymbol{\theta}^{(0)}) - \boldsymbol{\theta}^*\|^2$, all $\nabla \mathcal{L}$ cross-terms cancel, leaving $\phi_1 \leq \frac{L}{2} \|\boldsymbol{\theta}^{(0)} - \boldsymbol{\theta}^*\|^2$. Summing $\phi_k - \phi_{k-1} \leq 0$ over $k = 2, \dots, T$ and combining with the base yields $t_T^2 [\mathcal{L}(\boldsymbol{\theta}^{(T)}) - \mathcal{L}^*] \leq \frac{L}{2} \|\boldsymbol{\theta}^{(0)} - \boldsymbol{\theta}^*\|^2$. The first term of ϕ_T is $t_T^2 [\mathcal{L}(\boldsymbol{\theta}^{(T)}) - \mathcal{L}^*]$, and with $t_T \geq (T+1)/2$ this gives

$$\mathcal{L}(\boldsymbol{\theta}^{(T)}) - \mathcal{L}^* \leq \frac{\phi_1}{t_T^2} \leq \frac{2L \|\boldsymbol{\theta}^{(0)} - \boldsymbol{\theta}^*\|^2}{(T+1)^2}.$$

The mechanism: the β_k schedule is chosen *exactly so* the per-step decrease in ϕ_k is non-negative. Other schedules also work, notably $\beta_k = (k-1)/(k+2)$, which satisfies the same inequality and achieves the same $O(1/T^2)$ convergence rate.

5.5 Adaptive Learning Rate Methods

A single learning rate for all parameters is suboptimal: sparse features need larger updates; dense features need smaller ones. Adaptive methods adjust the learning rate per-parameter.

5.5.1 AdaGrad

AdaGrad [Duchi et al., 2011] adapts the learning rate per parameter by accumulating squared gradients:

$$G_j^{(t)} = G_j^{(t-1)} + \left(g_j^{(t)}\right)^2 \tag{5.30}$$

$$\theta_j^{(t+1)} = \theta_j^{(t)} - \frac{\eta}{\sqrt{G_j^{(t)} + \epsilon}} g_j^{(t)}, \tag{5.31}$$

where $g_j^{(t)} = \frac{\partial \mathcal{L}}{\partial \theta_j}$ and ϵ prevents division by zero.

INTUITION

Parameters with large cumulative squared gradients — frequently updated, high-curvature directions — get throttled to a smaller effective learning rate. Rare-feature parameters, which barely accumulate gradient, get to move faster. The denominator $\sqrt{G_j}$ acts as a cheap diagonal preconditioner.

COMMON MISTAKE

AdaGrad's $G_j^{(t)}$ accumulates monotonically throughout training. For long training runs the effective learning rate $\eta/\sqrt{G_j}$ shrinks to near zero, and learning stops. RMSProp fixes this.

5.5.2 RMSProp

RMSProp [Tieleman and Hinton, 2012] replaces the sum of squared gradients with an exponential moving average:

$$v_j^{(t)} = \beta_2 v_j^{(t-1)} + (1 - \beta_2) \left(g_j^{(t)}\right)^2 \quad (5.32)$$

$$\theta_j^{(t+1)} = \theta_j^{(t)} - \frac{\eta}{\sqrt{v_j^{(t)} + \epsilon}} g_j^{(t)}. \quad (5.33)$$

The EMA with factor β_2 (default 0.9 in Keras/TF but 0.99 in PyTorch) causes old squared gradients to decay, preventing the learning rate from going to zero. RMSProp fixes AdaGrad's decay problem with essentially this one change, and it was never published as a paper. The entire method traces back to slide 29 of a Geoff Hinton Coursera lecture. It later became the second-moment half of Adam.

5.5.3 Adam: Adaptive Moment Estimation

Adam [Kingma and Ba, 2015] combines momentum (first moment) with RMSProp (second moment), with bias correction:

DERIVATION · Adam: Full Derivation

At step t , given stochastic gradient $\mathbf{g}^{(t)}$:

1. First moment (momentum):

$$\mathbf{m}^{(t)} = \beta_1 \mathbf{m}^{(t-1)} + (1 - \beta_1) \mathbf{g}^{(t)} \quad (5.34)$$

2. Second moment (adaptive scale):

$$\mathbf{v}^{(t)} = \beta_2 \mathbf{v}^{(t-1)} + (1 - \beta_2) \mathbf{g}^{(t)} \odot \mathbf{g}^{(t)} \quad (5.35)$$

3. Bias correction (since $\mathbf{m}^{(0)} = \mathbf{v}^{(0)} = \mathbf{0}$, early estimates are biased toward zero):

$$\hat{\mathbf{m}}^{(t)} = \frac{\mathbf{m}^{(t)}}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}^{(t)} = \frac{\mathbf{v}^{(t)}}{1 - \beta_2^t} \quad (5.36)$$

4. Parameter update:

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \frac{\eta}{\sqrt{\hat{\mathbf{v}}^{(t)}} + \epsilon} \odot \hat{\mathbf{m}}^{(t)} \quad (5.37)$$

Why bias correction? Unrolling the first-moment EMA recurrence (starting from $\mathbf{m}^{(0)} = \mathbf{0}$):

$$\mathbf{m}^{(t)} = (1 - \beta_1) \sum_{k=1}^t \beta_1^{t-k} \mathbf{g}^{(k)}. \quad (5.38)$$

If gradients are stationary ($\mathbb{E}[\mathbf{g}^{(k)}] = \mathbb{E}[\mathbf{g}]$ for all k):

$$\mathbb{E}[\mathbf{m}^{(t)}] = (1 - \beta_1) \mathbb{E}[\mathbf{g}] \sum_{k=1}^t \beta_1^{t-k} = (1 - \beta_1) \mathbb{E}[\mathbf{g}] \cdot \frac{1 - \beta_1^t}{1 - \beta_1} = (1 - \beta_1^t) \mathbb{E}[\mathbf{g}]. \quad (5.39)$$

Hence $\hat{\mathbf{m}}^{(t)} = \mathbf{m}^{(t)} / (1 - \beta_1^t)$ satisfies $\mathbb{E}[\hat{\mathbf{m}}^{(t)}] = \mathbb{E}[\mathbf{g}]$: exactly unbiased. The same geometric-sum argument applies to the second moment with β_2 . At $t = 1$, the factor is $(1 - \beta_1)$; as $t \rightarrow \infty$ it approaches 1 (bias vanishes). For the first moment ($\beta_1 = 0.9$) the correction is negligible after ~ 45 steps; for the second moment ($\beta_2 = 0.999$) it remains $\sim 10\times$ at step 100 and $\sim 1.6\times$ at step 1,000, becoming negligible only after $\sim 4,600$ steps.

DERIVATION · Why Warmup, If $\hat{\mathbf{v}}^{(t)}$ Is Unbiased?

The bias correction fixes the *mean* of $\hat{\mathbf{v}}^{(t)}$; it does nothing about its *variance*, and early on the variance is the problem. Unrolling as above, with $\hat{\mathbf{v}}^{(t)} = \frac{1 - \beta_2}{1 - \beta_2^t} \sum_{k=1}^t \beta_2^{t-k} (\mathbf{g}^{(k)})^2$ and i.i.d. squared gradients of variance $\text{Var}[g^2]$

per coordinate,

$$\begin{aligned}\text{Var}[\hat{v}^{(t)}] &= \text{Var}[g^2] \cdot \frac{(1 - \beta_2)^2}{(1 - \beta_2^t)^2} \sum_{k=1}^t \beta_2^{2(t-k)} = \frac{\text{Var}[g^2]}{n_{\text{eff}}(t)}, \\ n_{\text{eff}}(t) &= \frac{(1 + \beta_2)(1 - \beta_2^t)}{(1 - \beta_2)(1 + \beta_2^t)}.\end{aligned}\tag{5.40}$$

At $t = 1$, $n_{\text{eff}} = 1$: the bias-corrected second moment is a *single-sample* estimate of $\mathbb{E}[g^2]$, exactly unbiased and wildly uncertain. For $t \ll 1/(1 - \beta_2)$ one has $n_{\text{eff}}(t) \approx t$, and it saturates at $(1 + \beta_2)/(1 - \beta_2)$ — about 2,000 samples for $\beta_2 = 0.999$, 39 for $\beta_2 = 0.95$.

Now the step. The update divides by $\sqrt{\hat{v}}$, and $v \mapsto v^{-1/2}$ is convex, so by Jensen $\mathbb{E}[1/\sqrt{\hat{v}}] > 1/\sqrt{\mathbb{E}[\hat{v}]}$; a second-order expansion makes the gap explicit,

$$\mathbb{E}\left[\frac{1}{\sqrt{\hat{v}(t)}}\right] \approx \frac{1}{\sqrt{\mathbb{E}[g^2]}} \left(1 + \frac{3}{8} \frac{\text{Var}[g^2]}{\mathbb{E}[g^2]^2} \cdot \frac{1}{n_{\text{eff}}(t)}\right).\tag{5.41}$$

The *effective step* is biased upward, by a factor that shrinks like $1/n_{\text{eff}}(t) \approx 1/t$ over the first few hundred steps. That is the case for warmup: not a biased $\hat{v}$, but a noisy one, feeding a convex $1/\sqrt{\cdot}$. Ramping η from zero over a horizon of order $1/(1 - \beta_2)$ steps keeps the product of the two factors bounded while n_{eff} grows, which is why warmup lengths of 10^3 – 10^4 steps pair with $\beta_2 = 0.999$, and why a lower β_2 tolerates a shorter ramp. (Liu et al. [2020] make this precise and derive an adaptive correction from it.)

DERIVATION · Choosing β_2 : Spike Response and Memory

The same EMA fixes two numbers, and between them they decide β_2 . Take a single coordinate at steady state with typical squared gradient $\bar{g}^2$, and hand it one anomalous batch whose gradient is $G \gg \bar{g}$. Set $\beta_1 = 0$ (RMSProp) for the spike analysis, so the numerator of the step is the raw gradient.

Spike response. One step of Equation (5.35) moves the second moment to

$$\hat{v} \approx (1 - \beta_2)G^2 + \beta_2\bar{g}^2 \approx (1 - \beta_2)G^2\tag{5.42}$$

(bias correction is negligible at steady state, so $\hat{v} \approx v$), and the denominator therefore rises only to $\sqrt{\hat{v}} \approx \sqrt{1 - \beta_2} G$ while the numerator rises all the way to G . A step scaled correctly to this gradient would divide G by G ; Adam

divides it by $\sqrt{1 - \beta_2} G$, so the step taken *on* the spike is inflated by

$$\frac{G}{\sqrt{1 - \beta_2} G} = \frac{1}{\sqrt{1 - \beta_2}} \quad (5.43)$$

relative to a correctly scaled one: $31.6\times$ at $\beta_2 = 0.999$ against $4.5\times$ at $\beta_2 = 0.95$. (Keeping the $\beta_2 \bar{g}^2$ term that Equation (5.42) drops, $\bar{g} = 1$ and $G = 100$ give 30.2 and 4.47.) With $\beta_1 = 0.9$ the first moment absorbs most of the spike, since $\hat{m}$ moves only to about $0.1G$: the largest step after the spike is then about $3.3\times$ normal at $\beta_2 = 0.999$ and no larger than normal at $\beta_2 = 0.95$, so the ordering survives while the size shrinks. Either way the largest steps of a training run follow its largest gradients, and β_2 decides by how much, which is one mechanism behind the loss spikes seen in large-scale Adam training.

Memory. The EMA averages over $\approx 1/(1 - \beta_2)$ steps, 1,000 at $\beta_2 = 0.999$ against 20 at 0.95, and that window is also how long the spike is remembered. Once the gradient returns to $\bar{g}$ the excess decays geometrically,

$$\hat{v}^{(t)} - \bar{g}^2 = \beta_2^t (\hat{v}^{(0)} - \bar{g}^2), \quad (5.44)$$

so with $\bar{g} = 1$ and $G = 100$ the denominator takes 3,862 steps at $\beta_2 = 0.999$, and 152 steps at $\beta_2 = 0.95$, to fall back within 10% of $\bar{g}$. Through every one of them the coordinate’s effective step is suppressed by a factor it did not earn: a rare token’s embedding, hit once by an enormous gradient, is then near-frozen for the next few thousand steps (Appendix E, Failure 3).

Both facts point the same way. Lower β_2 when the gradient distribution is heavy-tailed or non-stationary — long-sequence LLM pretraining, RL fine-tuning, any pipeline that occasionally serves a rare high-loss batch — and keep the 0.999 default when gradients are well behaved, where the longer window is free variance reduction.

Typical hyperparameters. $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ (in float32; use $\epsilon = 10^{-6}$ for float16, where 10^{-8} underflows to zero; bfloat16 shares float32’s exponent range, so a larger ϵ there is a stability choice, not an underflow fix). These defaults work surprisingly well across problems.

INTUITION

Adam is gradient descent with each parameter’s step size normalized by the size of its recent gradients. A parameter whose gradient keeps the same sign has $|\hat{m}| \approx \sqrt{\hat{v}}$ and takes steps close to the full η ; one whose gradient keeps flipping sign has $\hat{m}$ averaging toward zero while $\sqrt{\hat{v}}$ does not, so its steps shrink.

PRACTITIONER LENS

WHEN THIS MATTERS. The default optimizer for every Transformer model you will train, fine-tune, or RLHF. Adam’s per-parameter adaptive learning rate is what lets the embedding-layer gradients (tiny) and the attention-output gradients (huge) coexist under a single global η .

DEBUGGING SIGNAL. NaN losses in the first few steps almost always come from either ϵ too small (denominator underflow in fp16) or unnormalized inputs causing a gradient spike before the second moment v_t has time to grow. Warmup is the standard fix because it gives v_t time to estimate the true gradient scale: 500–2000 steps for fine-tuning and smaller models; LLM pretraining typically uses 1,000–10,000 steps. Skipping warmup and using the default $\beta_2 = 0.999$ on a Transformer are two of the most common ways this derivation gets ignored in practice; see Appendix E, Failures 3 and 16, for what each looks like when it happens in production.

PRODUCTION IMPACT. Adam triples total memory compared to vanilla SGD (weights alone): each parameter needs $\mathbf{m}$ and $\mathbf{v}$ in addition to $\mathbf{w}$ itself. In standard mixed-precision training, Adam’s m_t and v_t are stored in fp32 (4 bytes each), so a 70B-parameter model incurs $2 \times 4 \text{ B} \times 70 \text{ B} \approx 560 \text{ GB}$ of optimizer state. This is $2 \times$ the fp32 master-weight storage (280 GB) but $4 \times$ the bf16 working-weight storage (140 GB); the relevant comparison depends on which copy you count. This overhead is the main reason ZeRO-1 and 8-bit Adam [Dettmers et al., 2022b] exist. (The lower figure of $\approx 280 \text{ GB}$ arises when bf16 optimizer state is used, as in bf16-Adam variants; 8-bit Adam halves it again to $\approx 140 \text{ GB}$, but fp32 moments are the standard.)

CHALLENGE · Find the bug

`code/challenges/ch05_find_the_bug.py` contains a small Adam training loop on a benign convex regression problem. Loss should fall toward $\sim 5 \times 10^{-5}$ without ever exceeding its starting value. Instead it spikes to $\sim 10^4$ within five steps, then slowly recovers. One operator in the file is wrong — the bug is specifically in the Adam derivation you just read.

- Run the script; reproduce the divergence.
- Walk through the first iteration of the update by hand for a single parameter, comparing against the bias-correction box above.
- Identify the wrong line and write the one-character fix.
- Re-run and confirm the loss never exceeds its starting value and reaches the noise floor.

Solution + a worked walk-through of the divergence are in `code/challenges/ch05_find_the_bug_solution.md`. This is a real bug that has shipped to production training frameworks more than once.

COMMON MISTAKE · *Adam Does Not Always Converge*

Despite its empirical success, Adam *does not* converge to the optimum in general, even for convex objectives. Adam’s effective step in coordinate j is $\eta/\sqrt{\hat{v}_{j,t}}$. Because $\hat{v}_{j,t}$ is an EMA with a finite window, a rare large gradient inflates the denominator on the very step it arrives, so its own step is scaled down, and it is then forgotten within $\approx 1/(1 - \beta_2)$ steps, after which the frequent small gradients again receive a large effective learning rate. The counterexample of Reddi et al. [2018] constructs a gradient sequence where this non-monotone effective LR prevents convergence: the denominator is not guaranteed to be non-decreasing, so the frequent small gradients, which point the wrong way, out-vote the rare large one that points the right way, and the iterates converge to the worst point of the feasible set.

AMSGrad fix. Take the running maximum over the *raw* (*uncorrected*) second moments $v_j^{(t)}$ (Algorithm 2 of Reddi et al. 2018; note that their Algorithm 2 also uses the *uncorrected* first moment $m_j^{(t)}$ in the update, whereas we keep the bias-corrected $\hat{m}_j^{(t)}$ as most implementations do), then use that maximum in the denominator:

$$v_{j,\max}^{(t)} = \max\left(v_{j,\max}^{(t-1)}, v_j^{(t)}\right), \quad \theta_j^{(t+1)} = \theta_j^{(t)} - \frac{\eta}{\sqrt{v_{j,\max}^{(t)}} + \epsilon} \hat{m}_j^{(t)}. \quad (5.45)$$

The running maximum makes the denominator non-decreasing, which is the monotonicity property that Reddi et al. (2018) use to establish convergence.

AdamW (the practical standard for LLMs). Weight decay should be applied to the parameters directly, not to the gradient. Loshchilov and Hutter

[2019] show that the standard L_2 regularization in Adam is *not* equivalent to weight decay because the adaptive scaling changes the effective regularization per parameter. AdamW decouples weight decay:

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \left[\frac{\hat{\mathbf{m}}^{(t)}}{\sqrt{\hat{\mathbf{v}}^{(t)} + \epsilon}} + \lambda_{\text{wd}} \boldsymbol{\theta}^{(t)} \right] \quad (5.46)$$

where λ_{wd} is the weight decay coefficient. AdamW with $\lambda_{\text{wd}} \in [0.01, 0.1]$ is the default optimizer for GPT, Llama, and most modern large-scale models.

Remark 5.15 (Adam and Natural Gradient). Adam’s diagonal preconditioner $\text{diag}(\hat{\mathbf{v}}^{(t)})^{-1/2}$ is a heuristic approximation to the *Fisher information matrix* $\mathbf{F}(\boldsymbol{\theta}) = \mathbb{E}_{x \sim p_{\boldsymbol{\theta}}(x)} [\nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(x) \nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(x)^{\top}]$. The *natural gradient* $\mathbf{F}^{-1} \nabla \mathcal{L}$ is the steepest descent direction in the space of probability distributions (KL geometry). Replacing $\mathbf{F}$ with $\text{diag}(\hat{\mathbf{v}})$ gives diagonal natural gradient $\text{diag}(\hat{\mathbf{v}})^{-1} \nabla \mathcal{L}$; Adam’s $\text{diag}(\hat{\mathbf{v}})^{-1/2}$ is a square-root variant of this approximation, not an exact recovery. See Chapter 14 §14.3 for the full derivation.

5.5.4 Why Adam Beats SGD on Anisotropic Curvature

The hero problem of this chapter asks why one global step size trains a ResNet and stalls on a Transformer. The answer is a statement about the spectrum of the curvature, and on the model that carries all of its content (a quadratic whose eigenvalues span orders of magnitude) it is short enough to derive in full. Gradient descent’s step size is capped by the sharpest direction while its progress is set by the flattest one, so the ratio between them, the condition number, lands directly in the step count. Adam’s per-coordinate normalization cancels that ratio.

DERIVATION · *The Spectral Argument: Where κ Enters, and How Adam Removes It*
Work with the diagonal quadratic

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2} \sum_{i=1}^d \lambda_i \theta_i^2, \quad 0 < \lambda_{\min} \leq \lambda_i \leq \lambda_{\max}, \quad \kappa = \frac{\lambda_{\max}}{\lambda_{\min}}. \quad (5.47)$$

Diagonality costs nothing: for any $\mathbf{A} \succ \mathbf{0}$ the spectral theorem (Theorem 1.4) writes $\frac{1}{2} \boldsymbol{\theta}^{\top} \mathbf{A} \boldsymbol{\theta}$ in the form Equation (5.47) in the rotated coordinates $\mathbf{Q}^{\top} \boldsymbol{\theta}$, so the diagonal case is the general case seen in its own eigenbasis.

Gradient descent. Here $g_i = \lambda_i \theta_i$, so Equation (5.1) decouples into d independent scalar recursions

$$\theta_i^{(t+1)} = (1 - \eta \lambda_i) \theta_i^{(t)}, \quad (5.48)$$

and each coordinate contracts by its own factor $|1 - \eta \lambda_i|$ while one η serves all of them. Stability in the sharpest direction forces $|1 - \eta \lambda_{\max}| < 1$, that is $\eta < 2/\lambda_{\max}$, and at that ceiling the flattest coordinate contracts by no more than

$$|1 - \eta \lambda_{\min}| > 1 - \frac{2\lambda_{\min}}{\lambda_{\max}} = 1 - \frac{2}{\kappa}. \quad (5.49)$$

Shrinking that coordinate by a factor $1/\epsilon$ therefore costs $\ln(1/\epsilon)/[-\ln(1 - 2/\kappa)] \approx (\kappa/2) \ln(1/\epsilon)$ steps: $\Theta(\kappa)$, with the accuracy fixing the constant and the spectrum fixing everything else. This is Theorem 5.5's $(1 - 1/\kappa)^T$ read one coordinate at a time, and Equation (5.49) is the $\rho_{\text{GD}} = (\kappa - 1)/(\kappa + 1)$ of Equation (5.23) to leading order in $1/\kappa$.

Adam. Run Equation (5.37) full-batch, so the gradient is deterministic, and let the moments settle while the gradient varies slowly: $\hat{m}_i \rightarrow g_i$ and $\hat{v}_i \rightarrow g_i^2$, hence $\sqrt{\hat{v}_i} \rightarrow |g_i|$ and

$$\theta_i^{(t+1)} = \theta_i^{(t)} - \eta \frac{g_i}{|g_i|} = \theta_i^{(t)} - \eta \text{sign}(\lambda_i \theta_i^{(t)}). \quad (5.50)$$

The curvature has canceled. Every coordinate moves the same distance η per step whatever its λ_i , the flattest direction stops waiting for the sharpest, and crossing a coordinate of scale $|\theta_i|$ takes $|\theta_i|/\eta$ steps — a count with no κ anywhere in it. Adam in this limit is sign descent with momentum [Balles and Hennig, 2018].

Numbers. Take $d = 2$ with $\lambda = (1, 10^4)$, a spread comfortably inside what is measured on Transformer Hessians [Ghorbani et al., 2019, Zhang et al., 2024], so $\kappa = 10^4$; start both methods at $\theta^{(0)} = (1, 1)$ and stop when $|\theta_i| \leq 10^{-2}$ in *both* coordinates. Gradient descent at $\eta = 1.95 \times 10^{-4}$ (2.5% under $2/\lambda_{\max}$) needs 23,614 steps, against the $(\kappa/2) \ln 100 = 23,026$ that Equation (5.49) predicts. Adam with the defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$ needs 2,203 steps at $\eta = 10^{-3}$ and 784 at $\eta = 3 \times 10^{-3}$, roughly twice the ideal $|\theta_i|/\eta$, because $\hat{v}$ lags a shrinking gradient and over-damps the step. Then change the spectrum and rerun: at $\kappa = 1, 10^2$ and 10^4 gradient descent (again 2.5% under $2/\lambda_{\max}$) takes 90, 234 and 23,614 steps, while Adam at $\eta = 10^{-3}$ takes 2,203 in all three.

Sign descent is not a free lunch. Discarding $|g_i|$ is what makes Equation (5.50) indifferent to the condition number, and it is the same act that leaves the step size carrying all the information about how far to move: the update has magnitude η whether the coordinate sits a mile from its optimum or an inch, so Adam settles into a band of width $\sim\eta$ around the minimum and stays there until η is decayed. A schedule therefore does work for Adam that it does not have to do for SGD, supplying the magnitude information the normalization threw away, which is why every Adam recipe in this chapter arrives with warmup and cosine decay (Section 5.5.5) attached.

The second caveat is where the hero problem lands. When curvature is close to isotropic there is nothing for the normalization to cancel: at $\kappa = 1$ in the experiment above, gradient descent with $\eta = 0.5$ reaches the same tolerance in 7 steps while Adam at $\eta = 10^{-3}$ still spends 2,203, since a gradient step is proportional to the distance remaining and a sign step is not. Vision CNNs trained on batch-normalized features sit near that end of the scale; normalization is itself a preconditioner, and the measured Hessian spectra are comparatively benign [Ghorbani et al., 2019], which is why well-tuned SGD with momentum still wins on ResNet-50 and why the CNN row of the decision tree below recommends it. Transformers sit at the far end: embedding, attention and LayerNorm blocks carry gradient scales differing by orders of magnitude, and the Hessian inherits that spread as block-wise heterogeneity [Zhang et al., 2024], giving a κ large enough that Equation (5.49) is a wall. GPT-2 with SGD barely moves because its flat directions need $\Theta(\kappa)$ steps when they share a step size with the sharp ones; Adam needs $|\theta_i|/\eta$, independent of the ratio.

— OPEN QUESTION · *Why Does Adam Work So Well on Transformers?* —

The anisotropic curvature story is compelling: Transformer loss surfaces have gradients that vary by orders of magnitude across parameters (embedding layers vs. attention outputs), and Adam’s per-parameter scaling compensates where a uniform step size cannot. But this is a story, not a theorem. No result says “Adam converges to a δ -stationary point on Transformer-shaped losses faster than SGD with this quality bound.” Nonconvex guarantees for Adam and AMSGrad do exist (convergence to approximate stationary points at roughly $O(1/\sqrt{T})$ up to log factors, under smoothness and bounded-gradient assumptions), but they are worst-case bounds that do not separate Adam from SGD. μP explains how to transfer learning rates across model scales but not why Adam dominates other diagonal preconditioners. The empirical fact — that AdamW is robustly better than carefully tuned SGD on language models —

is not fully explained by any existing theory. Why this specific second-moment estimator generalizes as well as it does remains open.

5.5.5 Learning Rate Schedules

A constant learning rate rarely gives the best result with Adam. Warmup’s justification is not repeated here; it is the moment-estimator variance argument already worked out in Equations (5.40) and (5.41); what follows is just the schedule itself, paired with the decay that follows it in practice.

Linear warmup. For the first T_{warm} steps, ramp the learning rate linearly:

$$\eta_t = \eta_{\max} \cdot \frac{t}{T_{\text{warm}}}, \quad t < T_{\text{warm}}. \quad (5.51)$$

Cosine annealing. After warmup, decay the learning rate following a cosine curve:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\pi \frac{t - T_{\text{warm}}}{T - T_{\text{warm}}} \right) \right). \quad (5.52)$$

This smoothly decreases from $\eta_{\max}$ to $\eta_{\min}$ over the remaining $T - T_{\text{warm}}$ steps. Many recent LLM runs instead use warmup-stable-decay (WSD): hold $\eta_{\max}$ for most of training, then decay quickly over the final 10–20% of steps, which avoids committing to T in advance and makes it cheap to extend a run or branch continued pretraining from a mid-run checkpoint.

Figure 5.1 compares step decay, plain cosine annealing, and linear warmup followed by cosine decay on the same axes: the warmup-plus-decay curve is the one that avoids both a jagged, discontinuous schedule and a full-strength learning rate applied before the moment estimates have stabilized.

5.6 Hyperparameter Transfer Across Scale (μP)

A practical problem: you tune learning rates on a 125M-parameter model, then run a 70B-parameter model with the same recipe and it diverges. Standard parametrization makes the optimal learning rate depend on width: $\eta^* \sim 1/d$ for the largest weight matrices, $\eta^* \sim 1$ for biases, with no single multiplier that works at both widths. Each model size must be retuned, which is prohibitively expensive at the frontier.

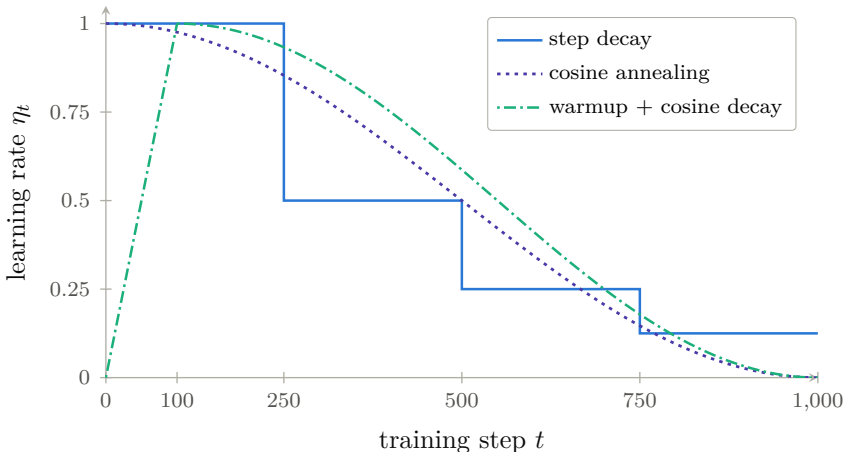


Figure 5.1: Three learning-rate schedules η_t over $T = 1,000$ training steps, normalized to $\eta_{\max} = 1$. **Step decay** holds η constant and halves it at fixed intervals, producing abrupt drops that can destabilize the moment estimates $\hat{m}_t, \hat{v}_t$ at each discontinuity. **Cosine annealing** (Equation (5.52) with $T_{\text{warm}} = 0$) decays smoothly from step one, but applies full-strength updates while the early gradient estimates are still noisy. **Linear warmup followed by cosine decay** (Equations (5.51) and (5.52), warmup over the first 10% of steps) avoids both failure modes: the ramp keeps early updates small while $\hat{m}_t, \hat{v}_t$ are unreliable, and the subsequent cosine decay anneals the step size for fine-grained convergence near the end of training. This is why warmup-plus-decay schedules dominate modern large-scale training: they match the learning rate to the time-varying reliability of the gradient signal, rather than holding it fixed or degrading it on a schedule blind to optimization dynamics.

The μP recipe [Yang et al., 2021]. *Maximal Update Parametrization* (μP) rescales the initialization and learning rate per-tensor so that the typical *activation update* $\Delta\mathbf{a}$ has $O(1)$ magnitude at every width. For Adam-trained Transformers with hidden width d , μP scales:

Tensor class	Init variance	Learning-rate multiplier
Input embedding	1	1
Hidden weight matrix ($d \times d$)	$1/d$	$1/d$
Output (readout) weight	$1/d^2$	$1/d$
Bias / LayerNorm gain	0 / 1	1

The factor $1/d$ on the Adam learning rate for hidden weights is the critical piece.

Under *standard parametrization* (SP), Adam normalizes per-parameter steps to $O(1)$, but the fan-in sum over d weights makes each neuron’s activation update scale as $d \cdot \eta$, an $O(d)$ blow-up that forces a width-dependent optimal LR. μP resolves this by applying the $1/d$ LR multiplier to hidden weights, restoring $O(1)$ activation updates at every width. The readout row matters just as much: μP shrinks the output-weight init variance to $1/d^2$ and scales the readout’s Adam LR by $1/d$ — equivalently, keeps a variance-1 init with an *unscaled* LR and multiplies the output logits by $1/d$ — so that the logits and their updates stay $O(1)$ as the hidden width grows (this is the “output logit scale $\propto 1/d$ ” rule in the decision tree below).

The transfer property. Under μP , the optimal hyperparameters become *width-independent*: the learning rate, momentum, init scale, and warmup that work for a small proxy also work for a model hundreds of times larger (Yang et al. [2021] transferred from a 40M-parameter proxy to the 6.7B-parameter GPT-3). The practical recipe:

1. Tune hyperparameters by sweeping a small proxy model (e.g. 40M params with $d = 256$) under μP .
2. Apply the same hyperparameters to the production model.
3. Save most of the hyperparameter-search compute: the proxy model can be orders of magnitude smaller than the production model, so the sweep cost is dominated by the proxy, not the target.

— PRACTITIONER LENS —

WHEN THIS MATTERS. Pretraining any model larger than your current tuning budget can support. Frontier labs use μP to avoid retuning end-to-end at every model size. (DeepNorm, Ch 8, addresses a separate concern, depth stability via $\alpha = (2N)^{1/4}$ residual scaling, and is not a variant of μP .)

DEBUGGING SIGNAL. Set up the experiment: train two model widths (d_1, d_2) with μP over a wide learning-rate grid; plot the loss-vs-LR curve. If the optimum coincides between widths, μP is wired correctly. If the optima differ by more than a factor of 2, some tensor class is in the wrong column of the table above.

PRODUCTION IMPACT. For a frontier-scale tune: each width sweep costs roughly 5–10 $\times$ the cost of a single training run at that width. Avoiding a full sweep at the largest scale by transferring from a 100 $\times$ -smaller proxy saves the bulk of the hyperparameter-search budget — millions of dollars on a flagship model.

5.7 Second-Order Methods

5.7.1 Newton’s Method

Instead of descending along the gradient, Newton’s method descends along the gradient *preconditioned by the Hessian*:

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \nabla^2 \mathcal{L}(\boldsymbol{\theta}^{(t)})^{-1} \nabla \mathcal{L}(\boldsymbol{\theta}^{(t)}). \quad (5.53)$$

DERIVATION · *Newton’s Method from a Quadratic Approximation*

Approximate $\mathcal{L}$ near $\boldsymbol{\theta}^{(t)}$ with its second-order Taylor expansion:

$$\mathcal{L}(\boldsymbol{\theta}) \approx \mathcal{L}(\boldsymbol{\theta}^{(t)}) + \nabla \mathcal{L}^\top \delta + \frac{1}{2} \delta^\top \nabla^2 \mathcal{L} \delta, \quad \delta = \boldsymbol{\theta} - \boldsymbol{\theta}^{(t)}. \quad (5.54)$$

This is a quadratic in δ . Setting $\partial/\partial\delta = 0$:

$$\nabla^2 \mathcal{L} \delta = -\nabla \mathcal{L} \implies \delta = -\nabla^2 \mathcal{L}^{-1} \nabla \mathcal{L}. \quad (5.55)$$

Adding to $\boldsymbol{\theta}^{(t)}$ gives Equation (5.53).

Newton’s method converges *quadratically* near the solution ($\|\boldsymbol{\theta}^{(t+1)} - \boldsymbol{\theta}^*\| \leq C \|\boldsymbol{\theta}^{(t)} - \boldsymbol{\theta}^*\|^2$), provided $\nabla^2 \mathcal{L}(\boldsymbol{\theta}^*)$ is positive definite, $\nabla^2 \mathcal{L}$ is Lipschitz-continuous near $\boldsymbol{\theta}^*$, and the initial point $\boldsymbol{\theta}^{(0)}$ is sufficiently close to $\boldsymbol{\theta}^*$; compared to linear convergence for gradient descent. However, inverting the $d \times d$ Hessian costs $O(d^3)$, making it infeasible for the millions of parameters in neural networks.

5.7.2 Why the Exact Method Dies at Scale

The obstruction here is arithmetic, not subtlety. The Hessian has d^2 entries and the Newton solve $\nabla^2 \delta = -\nabla \mathcal{L}$ costs $O(d^3)$ flops. For a 7B-parameter model ($d = 7 \times 10^9$), the Hessian has $d^2 = 4.9 \times 10^{19}$ entries; stored in fp32 (4 bytes each) that is 1.96×10^{20} bytes $\approx$ **196 exabytes** — roughly the storage of a large cloud region, for *one* curvature matrix that changes every step. (For comparison, the gradient itself is $7 \times 10^9 \times 4$ bytes = 28 GB.) The direct solve would cost $d^3 \approx 3.4 \times 10^{29}$ flops — about ten *millennia* on an exaflop (10^{18} flop/s) cluster, per iteration. There is also a statistical obstruction: with minibatch gradients, the Hessian estimate is noisy, and Newton’s quadratic convergence guarantee assumes exact derivatives; second-order information is far harder to estimate from samples than first-order. Every practical “second-order” method is therefore a bet on

structure: which cheap approximation of ∇^2 retains enough curvature to be worth its overhead?

5.7.3 L-BFGS (Quasi-Newton)

L-BFGS approximates $\nabla^{2^{-1}}$ using a limited history of gradient differences, requiring only $O(md)$ storage for m recent gradient pairs. It is the optimizer of choice for small-to-medium scale convex problems (e.g., full-batch logistic regression).

The mechanism: after each step, store the pair $\mathbf{s}_t = \boldsymbol{\theta}^{(t+1)} - \boldsymbol{\theta}^{(t)}$, $\mathbf{y}_t = \nabla \mathcal{L}(\boldsymbol{\theta}^{(t+1)}) - \nabla \mathcal{L}(\boldsymbol{\theta}^{(t)})$. By Taylor expansion $\mathbf{y}_t \approx \nabla^2 \mathbf{s}_t$; each pair is one directional probe of the curvature (the *secant condition*). BFGS builds an inverse-Hessian estimate satisfying all past secant conditions via a rank-two update per step; *limited-memory* BFGS keeps only the last m pairs (typically $m = 5\text{--}20$) and applies the implied rank- $2m$ correction to a scaled identity through the two-loop recursion, at $O(md)$ cost per step and never forming a matrix. For $d = 10^6$ and $m = 20$ that is 160 MB of history — trivial. L-BFGS wins when its assumptions hold: **full-batch**, **deterministic** gradients (secant pairs computed from noisy minibatch gradients encode noise, not curvature, and can catastrophically corrupt the estimate [Bottou et al., 2018]), smooth objectives, and $d \lesssim 10^6$. That profile fits classical ML (logistic regression, CRFs, style-transfer image optimization, physics-informed losses) and fine-tuning small heads, not stochastic pretraining.

5.7.4 The Practical Ladder: Diagonal and Block Curvature

Between “no curvature” (SGD) and “all of it” (Newton) sits a ladder of structured approximations, ordered by how much of ∇^2 they keep.

Diagonal. Keep only d numbers. Adam’s second-moment EMA $\hat{\mathbf{v}}_t$ (Section 5.5) is this move: a per-parameter curvature proxy (squared-gradient scale standing in for ∇_{ii}^2), which is why Adam is best understood as diagonal preconditioning rather than as a learning-rate schedule.

Block / Kronecker-factored. Keep curvature *within* each layer but factor it. K-FAC [Martens and Grosse, 2015] approximates each layer’s Fisher block as a Kronecker product $\mathbf{A} \otimes \mathbf{G}$ of two small matrices (input-activation and gradient covariances), so inversion costs the cube of the layer width rather than of d . Shampoo [Gupta et al., 2018] preconditions each weight matrix on both sides with per-dimension curvature roots, a similar Kronecker bet with cheaper bookkeeping; distributed variants have won large-scale training-efficiency benchmarks, making this the most production-credible rung above diagonal. Its best-known recent descendant, Muon, drops the accumulated statistics and orthogonalizes each weight

matrix’s momentum with a few Newton–Schulz iterations; it has been used to pretrain trillion-parameter MoE models (Kimi K2) and is the non-Adam optimizer a reader is most likely to meet in current LLM recipes.

These Fisher-based preconditioners are best understood as approximations to the *natural gradient*, which replaces the Euclidean geometry of parameter space with the KL geometry of the model’s predictions — the information-geometric view developed in Section 14.3.

PRACTITIONER LENS

WHEN THIS MATTERS. Fine-tuning a small model or fitting a classical convex model with full-batch gradients and $d \lesssim 10^6$: use L-BFGS (`scipy.optimize.minimize(method="L-BFGS-B")` or `torch.optim.LBFGS`) and enjoy convergence in tens of iterations. Stochastic pretraining at scale: use an adaptive first-order method (AdamW), i.e. the diagonal rung of the ladder; evaluate Shampoo-class methods only when you can amortize their per-step overhead against measured wall-clock savings.

DEBUGGING SIGNAL. Newton or a quasi-Newton method *diverging* usually traces back to an indefinite ∇^2 : away from a minimum the quadratic model has saddle directions, and $-\nabla^{2-1}\nabla\mathcal{L}$ can point *uphill*. The standard fixes: damping (solve with $\nabla^2 + \tau\mathbf{I}$, which interpolates toward gradient descent as τ grows — the Levenberg–Marquardt idea) or a trust region that caps $\|\delta\|$. When `torch.optim.LBFGS` explodes on minibatches, the culprit is rarely your code; the secant condition is being fed noise, so switch to full-batch or to Adam.

PRODUCTION IMPACT. Frontier labs still ship first-order optimizers because the relevant metric is loss per *wall-clock dollar*, not per iteration. Adam’s diagonal costs $O(d)$ and parallelizes perfectly; each rung up the ladder buys better-conditioned steps but pays in memory, communication, and implementation risk at 10^4 -GPU scale, and the $\sqrt{\kappa}$ -style gains shrink when gradient noise, not curvature, is the binding constraint. At pretraining scale the diagonal usually offers the best exchange rate.

5.8 Constrained Optimization: Lagrangians and KKT

INTUITION

Plenty of ML problems come with hard constraints bolted on: minimize the objective, subject to these limits. Lagrange multipliers turn the constrained problem into an unconstrained one by pricing constraint violations as a penalty. The KKT conditions are what tell you whether you've actually found the solution or just a plausible guess.

5.8.1 Equality Constraints: Lagrangians

Problem: $\min_{\mathbf{x}} f(\mathbf{x})$ s.t. $h_i(\mathbf{x}) = 0$, $i = 1, \dots, m$.

Form the *Lagrangian*:

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\nu}) = f(\mathbf{x}) + \sum_{i=1}^m \nu_i h_i(\mathbf{x}). \quad (5.56)$$

At the optimum, $\nabla_{\mathbf{x}} \mathcal{L} = 0$ and $h_i(\mathbf{x}) = 0$ for all i .

5.8.2 Inequality Constraints: KKT Conditions

Problem: $\min_{\mathbf{x}} f(\mathbf{x})$ s.t. $g_i(\mathbf{x}) \leq 0$, $i = 1, \dots, m$; $h_j(\mathbf{x}) = 0$, $j = 1, \dots, p$.

Lagrangian: $\mathcal{L}(\mathbf{x}, \boldsymbol{\alpha}, \boldsymbol{\nu}) = f(\mathbf{x}) + \sum_i \alpha_i g_i(\mathbf{x}) + \sum_j \nu_j h_j(\mathbf{x})$.

Theorem 5.16 (KKT Conditions). *Under the Linear Independence Constraint Qualification (LICQ), that the gradients of active inequality constraints and all equality constraints are linearly independent at $\mathbf{x}^*$, the KKT conditions are necessary at a local minimum $\mathbf{x}^*$:*

$$\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}^*, \boldsymbol{\alpha}^*, \boldsymbol{\nu}^*) = \mathbf{0} \quad (\text{stationarity}) \quad (5.57)$$

$$g_i(\mathbf{x}^*) \leq 0 \quad \forall i \quad (\text{primal feasibility}) \quad (5.58)$$

$$h_j(\mathbf{x}^*) = 0 \quad \forall j \quad (\text{equality constraints}) \quad (5.59)$$

$$\alpha_i^* \geq 0 \quad \forall i \quad (\text{dual feasibility}) \quad (5.60)$$

$$\alpha_i^* g_i(\mathbf{x}^*) = 0 \quad \forall i \quad (\text{complementary slackness}) \quad (5.61)$$

Remark 5.17. The KKT conditions are necessary only under a **constraint qualification (CQ)**. For convex programs (convex objective, convex inequality constraints, affine equality constraints), **Slater's condition** is the standard CQ: if there exists a strictly feasible point $\mathbf{x}_0$ with $g_i(\mathbf{x}_0) < 0$ for all inequality constraints, then (i)

strong duality holds (zero duality gap), and (ii) KKT conditions are *necessary* for optimality at any minimizer. Under convexity of f and g_i and h_j affine, KKT conditions are also *sufficient* for optimality, regardless of whether a CQ holds. Strong duality requires Slater's condition separately; KKT sufficiency does not. Without a CQ, KKT conditions may fail to hold at the optimum even if one exists.

Proof sketch (necessity). At $\mathbf{x}^*$, primal feasibility holds by assumption. Stationarity follows from the first-order optimality condition on the Lagrangian: moving $\mathbf{x}$ slightly along any feasible direction $\mathbf{d}$ must not decrease f , so $\nabla f(\mathbf{x}^*) + \sum_i \alpha_i \nabla g_i(\mathbf{x}^*) + \sum_j \nu_j \nabla h_j(\mathbf{x}^*) = \mathbf{0}$. Dual feasibility $\alpha_i \geq 0$ is required so that tightening an inequality constraint (making g_i more negative) cannot decrease the multiplier-weighted penalty; i.e., the multiplier pushes *against* the constraint, not toward it. Complementary slackness: if $g_i(\mathbf{x}^*) < 0$ (constraint is slack), the constraint does not restrict any small move away from $\mathbf{x}^*$, so it can be dropped from the local problem and its multiplier set to $\alpha_i^* = 0$; if $g_i(\mathbf{x}^*) = 0$ (active), α_i^* may be positive. Hence $\alpha_i^* g_i(\mathbf{x}^*) = 0$ in either case. (Sufficiency holds when f and all g_i are convex and h_j are affine: KKT conditions then guarantee global optimality.) $\square$

INTUITION

Complementary slackness Equation (5.61) is the key condition: either the constraint g_i is inactive ($g_i(\mathbf{x}^*) < 0$, in which case $\alpha_i^* = 0$ — it doesn't affect the solution), or the constraint is active ($g_i(\mathbf{x}^*) = 0$), in which case $\alpha_i^* \geq 0$ (and $\alpha_i^* > 0$ in non-degenerate cases), meaning the multiplier pushes the solution onto the constraint boundary. In SVMs, active constraints correspond to support vectors.

Example 5.18 (Solving a Constrained Quadratic with KKT). Minimize $f(x, y) = (x - 2)^2 + (y - 1)^2$ subject to $g_1(x, y) = x + y - 1 \leq 0$ and $g_2(x, y) = -x \leq 0$. The unconstrained minimizer of f is $(2, 1)$, which violates g_1 ($2 + 1 - 1 = 2 > 0$), so the constraint is active at the solution.

Stationarity. With multipliers $\alpha_1, \alpha_2 \geq 0$ for g_1, g_2 , Equation (5.57) reads $\nabla f + \alpha_1 \nabla g_1 + \alpha_2 \nabla g_2 = \mathbf{0}$, i.e.

$$\begin{aligned} 2(x - 2) + \alpha_1 - \alpha_2 &= 0, \\ 2(y - 1) + \alpha_1 &= 0. \end{aligned}$$

Guess the active set. Try g_1 active ($g_1 = 0$) and g_2 inactive ($\alpha_2 = 0$, expecting $x > 0$). The system becomes

$$2(x - 2) + \alpha_1 = 0, \quad 2(y - 1) + \alpha_1 = 0, \quad x + y - 1 = 0,$$

three linear equations in x, y, α_1 . Subtracting the first two gives $x - 2 = y - 1$, i.e. $x = y + 1$; substituting into $x + y = 1$ gives $(y + 1) + y = 1$, so $y^* = 0$, $x^* = 1$, and then $\alpha_1^* = -2(x^* - 2) = 2$.

Check the candidate $(x^*, y^*, \alpha_1^*, \alpha_2^*) = (1, 0, 2, 0)$.

- *Primal feasibility*: $g_1(1, 0) = 1 + 0 - 1 = 0 \leq 0$ and $g_2(1, 0) = -1 \leq 0$. Both hold (g_2 strictly, so $x^* > 0$ as assumed).
- *Dual feasibility*: $\alpha_1^* = 2 \geq 0$ and $\alpha_2^* = 0 \geq 0$. Both hold.
- *Complementary slackness*: $\alpha_1^* g_1(1, 0) = 2 \cdot 0 = 0$ and $\alpha_2^* g_2(1, 0) = 0 \cdot (-1) = 0$. Both hold.
- *Stationarity, by direct substitution*: $2(1 - 2) + 2 - 0 = -2 + 2 = 0$ and $2(0 - 1) + 2 = -2 + 2 = 0$. Both equations hold exactly.

Every KKT condition is satisfied at $(1, 0)$ with $\alpha_1^* = 2, \alpha_2^* = 0$, so it is a KKT point. Geometrically, the constraint g_1 pins the solution onto the line $x + y = 1$ (the closest point on that line to the unconstrained optimum $(2, 1)$), while g_2 never binds since the optimum already sits well inside $x \geq 0$.

From KKT point to global minimizer. f is convex (a sum of squares) and g_1, g_2 are affine, so Slater's condition holds trivially (e.g. $(0.5, 0)$ satisfies $g_1(0.5, 0) = -0.5 < 0$ and $g_2(0.5, 0) = -0.5 < 0$ strictly), and by the KKT-sufficiency fact noted in Theorem 5.17 (convex f , convex g_i , affine $h_j \Rightarrow$ KKT implies global optimality), the KKT point $(x^*, y^*) = (1, 0)$ is not merely a candidate but the actual global minimizer, with $f(1, 0) = (1 - 2)^2 + (0 - 1)^2 = 2$.

PRACTITIONER LENS

WHEN THIS MATTERS. Whenever your training loss has a hard constraint — KL budget in RLHF, trust-region radius in PPO, weight-norm constraints, fairness constraints. The Lagrangian formulation tells you how to convert each constraint to a penalty term and how to update the multipliers α_i online.

DEBUGGING SIGNAL. If the multiplier α blows up, the constraint is fighting the objective harder than your optimizer can absorb — either soften the constraint or reduce its weight. If $\alpha \rightarrow 0$, the constraint is inactive and you can remove it entirely.

PRODUCTION IMPACT. In RLHF the KL constraint is the safety rail that keeps the policy near the reference model. Choosing β (the multiplier) is one of the most important choices in the RL stage; DPO (Chapter 13) replaces online RL by analytically solving the constrained objective and reparameterizing it as a binary classification loss, eliminating the rollout loop and learned reward

model, but DPO retains its own β that must still be tuned.

5.8.3 Lagrangian Duality

The *Lagrange dual function*:

$$d(\boldsymbol{\alpha}, \boldsymbol{\nu}) = \inf_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\alpha}, \boldsymbol{\nu}). \quad (5.62)$$

d is always concave (infimum of affine functions in $\boldsymbol{\alpha}, \boldsymbol{\nu}$). The *dual problem* is $\max_{\boldsymbol{\alpha} \geq 0, \boldsymbol{\nu}} d(\boldsymbol{\alpha}, \boldsymbol{\nu})$.

Remark 5.19 (Weak Duality, and Budget versus Penalty). For any $\boldsymbol{\alpha} \succeq \mathbf{0}$, any $\boldsymbol{\nu}$, and any primal-feasible $\mathbf{x}$, the penalty terms obey $\sum_i \alpha_i g_i(\mathbf{x}) \leq 0$ and $\sum_j \nu_j h_j(\mathbf{x}) = 0$, so $\mathcal{L}(\mathbf{x}, \boldsymbol{\alpha}, \boldsymbol{\nu}) \leq f(\mathbf{x})$; and the infimum over all $\mathbf{x}$ in Equation (5.62) is no larger than the value at that particular one. Hence $d(\boldsymbol{\alpha}, \boldsymbol{\nu}) \leq f(\mathbf{x})$ for every feasible $\mathbf{x}$, and minimizing over them,

$$d(\boldsymbol{\alpha}, \boldsymbol{\nu}) \leq p^* := \min\{f(\mathbf{x}) : g_i(\mathbf{x}) \leq 0, h_j(\mathbf{x}) = 0\} \quad \text{for all } \boldsymbol{\alpha} \succeq \mathbf{0}. \quad (5.63)$$

Nothing in that argument assumes convexity. With the concavity of d noted above, the dual is always a concave maximization whose value d^* underestimates p^* by the *duality gap* $p^* - d^* \geq 0$, and Slater's condition (Theorem 5.17) is the standard sufficient condition for the gap to be zero.

The consequence the book leans on is the equivalence of a budget and a penalty. Take a single constraint g and a weight $\beta \geq 0$, and let $\mathbf{x}_\beta$ minimize the penalized objective $f + \beta g$. For any $\mathbf{x}$ with $g(\mathbf{x}) \leq g(\mathbf{x}_\beta)$,

$$f(\mathbf{x}) \geq f(\mathbf{x}_\beta) + \beta(g(\mathbf{x}_\beta) - g(\mathbf{x})) \geq f(\mathbf{x}_\beta), \quad (5.64)$$

so $\mathbf{x}_\beta$ solves the constrained problem at whatever budget $\epsilon = g(\mathbf{x}_\beta)$ it happens to spend, again with no convexity assumed. The converse needs strong duality: if it holds at budget ϵ with optimal multiplier β^* , then $p^* = \min_{\mathbf{x}} [f(\mathbf{x}) + \beta^*(g(\mathbf{x}) - \epsilon)]$, so the constrained solution minimizes $f + \beta^*g$. Under Slater's condition the two families coincide — sweeping β over $[0, \infty)$ traces exactly the solutions of “minimize f subject to $g \leq \epsilon$ ” as ϵ ranges over the attainable budgets, β decreasing as ϵ grows. Without it the sweep can skip budgets that no β reproduces.

Theorem 5.20 (Strong Duality). *For convex f and g_i , linear h_j , Slater's condition (existence of a strictly feasible point), and a finite primal optimal value $p^* \in \mathbb{R}$ (the feasible set is non-empty and f is bounded below on it), the duality gap is zero:*

$$\min_{\mathbf{x}: g_i(\mathbf{x}) \leq 0, h_j(\mathbf{x}) = 0} f(\mathbf{x}) = \max_{\boldsymbol{\alpha} \geq 0, \boldsymbol{\nu}} d(\boldsymbol{\alpha}, \boldsymbol{\nu}). \quad (5.65)$$

5.9 Non-Convex Landscapes in Deep Learning

Neural network loss surfaces are non-convex: they have many local minima, saddle points, and flat regions. Yet in practice SGD finds very good solutions. Why?

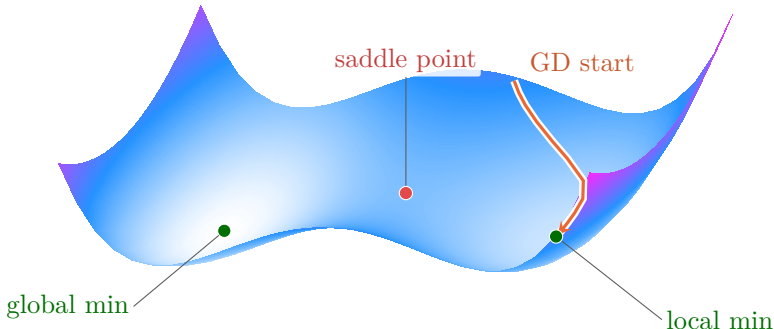


Figure 5.2: A non-convex loss surface $\mathcal{L}(\theta_1, \theta_2)$ illustrating the three landmark features of deep-learning optimization: a **saddle point** (gradient zero, indefinite Hessian), a **local minimum**, and a lower **global minimum**. Gradient descent (**orange**), started on the right-hand slope, settles into the local minimum even though the global one is lower: which basin it reaches depends on initialization. Near saddle points gradient descent slows down; momentum and second-order methods can escape them faster.

Figure 5.2 illustrates the three landmark features: a saddle point, a local minimum, and the global minimum, together with a typical gradient descent trajectory.

Saddle points. A point θ^* is a saddle point if $\nabla \mathcal{L}(\theta^*) = \mathbf{0}$ but the Hessian has both positive and negative eigenvalues. In high dimensions, saddle points are *far more common* than local minima. A crude but instructive model makes this quantitative: if each of the d Hessian eigenvalues at a critical point is positive or negative independently with probability $\frac{1}{2}$ (no reason favors one sign over the other away from special points), the point is a true local min or max only when all d signs agree,

$$P(\text{all } d \text{ eigenvalues same sign}) = 2 \cdot \left(\frac{1}{2}\right)^d = 2^{1-d}. \quad (5.66)$$

For $d = 100$ (a tiny network by modern standards), that is $2^{-99} \approx 1.6 \times 10^{-30}$: past a few dozen parameters, a random critical point is a saddle for all practical purposes.

The SGD-noise picture. Stochastic noise from minibatch sampling helps escape saddle points and sharp local minima, and appears to bias SGD toward flat minima with small curvature. Flatter minima are often argued to generalize better: empirically, large-batch training tends to converge to sharper minima with worse test performance [Keskar et al., 2017]. The link is not settled, however — common sharpness measures are not invariant to reparametrization, and Dinh et al. [2017] show that a sharp minimum can be rescaled into an arbitrarily flat one computing the same function, so flatness alone cannot fully explain generalization.

Remark 5.21 (The Saddle Escape Rate). “Natural” escape has a rate. Linearize gradient descent near a saddle along the eigenvector of the Hessian’s most negative eigenvalue $\lambda_{\min} < 0$: to first order the dynamics along that one direction is the scalar recursion $x_{t+1} = (1 - \eta\lambda_{\min})x_t = (1 + \eta|\lambda_{\min}|)x_t$, geometric growth away from the saddle. An initial displacement of size δ off the saddle — supplied by gradient noise, or by the point not being an exact critical point — therefore reaches $O(1)$ scale after

$$T_{\text{escape}} = \frac{\log(1/\delta)}{\log(1 + \eta|\lambda_{\min}|)} \approx \frac{\log(1/\delta)}{\eta|\lambda_{\min}|} \quad (5.67)$$

steps, the approximation using $\log(1 + x) \approx x$ for small $\eta|\lambda_{\min}|$: at $\eta = 0.01$, $|\lambda_{\min}| = 1$, $\delta = 10^{-6}$ the exact recursion gives $T_{\text{escape}} \approx 1388$ steps against 1382 from the approximation, half a percent apart. The dependence on δ is only logarithmic: shrinking the initial perturbation by ten orders of magnitude costs only a few thousand extra steps, which is the quantitative version of “gradient noise escapes saddles naturally.”

KEY IDEA

The practical lesson: trust SGD. For overparameterized networks, there is evidence that most local minima are nearly as good as the global minimum — proved by Choromanska et al. [2015] for a spin-glass model of the loss surface under strong simplifying assumptions (independent activation paths, among others), and observed empirically well beyond that setting — and saddle points are escaped naturally by gradient noise.

HOT TAKE

Shipping a training run without warmup in 2026 is malpractice. The math is on your side, though not in the way it is usually told: Adam’s second-moment estimate $\hat{v}_t$ is *unbiased* from step one (that is what bias correction buys), but at step one it is a single-sample estimate, and dividing by the square root of a

noisy estimate inflates the effective learning rate by a factor that only decays like $1/t$ (Equation (5.41)) — exactly when the optimizer is least equipped to absorb it. Linear warmup over a horizon of order $1/(1 - \beta_2)$ steps, followed by cosine decay to 10% of the peak, is the default; if you are deviating from this you should be able to defend it on the whiteboard.

Chapter Summary

KEY IDEA · Chapter Summary

Key results: (1) GD on L -smooth convex functions converges at $O(1/T)$; Nesterov achieves the optimal $O(1/T^2)$ by using a momentum sequence. (2) Adam adapts learning rates per parameter using running estimates of first and second moments; bias correction ensures $\hat{m}_t \approx \mathbb{E}[g_t]$ in early iterations. (3) KKT conditions are necessary for optimality under a constraint qualification (Slater’s condition for convex programs). (4) The PL condition is weaker than strong convexity but still implies linear convergence of GD.

Running example: On the unstandardized exam-score logistic regression, GD at $\eta = 0.01$ needs about 2.6×10^5 steps to close within 1% of optimal; Adam at $\eta = 0.1$ does it in ≈ 800 , because its per-coordinate scaling compensates for the intercept/ x_1/x_2 scale mismatch a single constant step size cannot.

Coming next: Chapter 6 shows how kernel methods extend linear models to infinite-dimensional feature spaces, and introduces Gaussian processes as Bayesian kernel regressors.

DECISION GUIDE · Which Optimizer?

Setting	Optimizer and key defaults
Transformer (any modality)	AdamW: $\beta_1=0.9$, $\epsilon=10^{-6}$ (bf16), clip 1.0, weight decay 0.1 (exclude 1D params: LayerNorm scales/biases and all biases). $\beta_2 = 0.95$ for long training runs: lower β_2 makes the second-moment estimate more responsive to sudden gradient surges, suppressing loss spikes at the cost of slightly noisier estimates; $\beta_2 = 0.999$ gives smoother but potentially less spike-stable training. (Section 5.5; the $1/\sqrt{1-\beta_2}$ spike inflation and the $1/(1-\beta_2)$ memory window are Equations (5.43) and (5.44))
CNN / ResNet	SGD + Nesterov momentum 0.9, cosine schedule, weight decay 10^{-4} . Batch size ≤ 256 ; linear LR scaling for larger batches. (Section 5.3)
Scaling from a known base shape	μ P: learning rate $\propto 1/d$, output logit scale $\propto 1/d$. Tune on a tiny proxy model; transfer to full scale. (Section 5.6)
Convex or small model ($d \lesssim 10^6$)	L-BFGS (quasi-Newton): fast linear convergence with a much better constant than GD, little tuning. Gradient storage: $2m$ vectors for memory- m variant. (Section 5.7)
Constrained problem (fairness, capacity)	KKT conditions + dual; projected gradient for simple sets. Duality gap = 0 when Slater's condition holds. (Section 5.8)

Exercises

Exercise 5.22 (Optimizer Comparison on a Non-Convex Landscape). This exercise compares gradient descent (GD), SGD with momentum, Adam, and Nesterov's method on a challenging non-convex function.

- (a) **Setup.** Define the 2D Rosenbrock function $f(x, y) = (1 - x)^2 + 100(y - x^2)^2$. Its unique minimum is at $(1, 1)$ with $f(1, 1) = 0$. Implement all four optimizers from scratch in NumPy (no autograd). Use initialization $(x_0, y_0) = (-1, 1)$ and run for $T = 5000$ steps.

- (b) **Convergence plots.** Use learning rates: GD $\eta = 10^{-3}$, momentum $\eta = 10^{-3}$ with $\beta = 0.9$, Adam $\eta = 10^{-2}$ with $\beta_1 = 0.9, \beta_2 = 0.999, \varepsilon = 10^{-8}$, Nesterov $\eta = 10^{-3}$ with $\beta = 0.9$. Plot $\log_{10}(f(x^{(t)}, y^{(t)}))$ vs. step t for all four on the same axes.
- (c) **Learning rate sensitivity.** For Adam, vary $\eta \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}\}$. Plot final loss at $T = 5000$ vs. $\log_{10}(\eta)$. Identify the optimal learning rate and explain why Adam is less sensitive to η than GD (relate to the adaptive scaling $1/\sqrt{\hat{v}_t}$).
- (d) **Gradient clipping.** Run GD with $\eta = 10^{-2}$ (too large) without and with gradient norm clipping at $\tau = 1.0$. Report whether clipping rescues convergence. Verify: the clipping operation $\tilde{g} = g \cdot \min(1, \tau/\|g\|)$ is the projection of g onto the ball $\{\|g\| \leq \tau\}$.

Exercise 5.23 (Learning Rate Schedules). (a) Implement cosine annealing with linear warmup: $\eta(t) = \eta_{\max} \cdot \frac{t}{T_{\text{warm}}}$ for $t \leq T_{\text{warm}}$, and $\eta(t) = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2} \left(1 + \cos\left(\pi \frac{t - T_{\text{warm}}}{T - T_{\text{warm}}}\right)\right)$ for $t > T_{\text{warm}}$ (matching Equation (5.52); use $\eta_{\min} = 0$ if no floor is needed).

- (b) With $T = 1000$, $T_{\text{warm}} = 100$, $\eta_{\max} \in \{10^{-1}, 10^{-2}, 10^{-3}\}$, plot the schedule $\eta(t)$ for $t = 0, \dots, T$.
- (c) Run Adam with this schedule on the exam-score logistic regression problem from Chapter 4. Compare final cross-entropy loss to constant $\eta = 10^{-3}$. Explain why warmup helps (the bias-corrected second moment $\hat{v}_t$ is a few-sample estimate early on, and dividing by its square root inflates the step; see Equations (5.40) and (5.41)).

Exercise 5.24 (Derivation). Prove the Adam bias correction: show that without the division by $(1 - \beta_1^t)$, the expected value of $\mathbf{m}^{(t)}$ is $(1 - \beta_1^t)\mathbb{E}[\mathbf{g}]$, not $\mathbb{E}[\mathbf{g}]$.

SOLUTION

The EMA update unrolls as $\mathbf{m}^{(t)} = (1 - \beta_1) \sum_{k=1}^t \beta_1^{t-k} \mathbf{g}^{(k)}$. Taking expectations (assuming $\mathbb{E}[\mathbf{g}^{(k)}] = \mathbb{E}[\mathbf{g}]$ for all k):

$$\mathbb{E}[\mathbf{m}^{(t)}] = (1 - \beta_1)\mathbb{E}[\mathbf{g}] \sum_{k=1}^t \beta_1^{t-k} = (1 - \beta_1)\mathbb{E}[\mathbf{g}] \cdot \frac{1 - \beta_1^t}{1 - \beta_1} = (1 - \beta_1^t)\mathbb{E}[\mathbf{g}].$$

Without correction, $\mathbf{m}^{(t)}$ underestimates the true gradient mean by factor $(1 - \beta_1^t)$. Dividing by $(1 - \beta_1^t)$ gives the unbiased estimate $\hat{\mathbf{m}}^{(t)} = \mathbf{m}^{(t)}/(1 - \beta_1^t)$. $\square$

Exercise 5.25 (Derivation). Prove that the Lagrange dual function $d(\boldsymbol{\alpha}) = \inf_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\alpha})$ is always concave in $\boldsymbol{\alpha}$, even when f is non-convex. (Hint: the infimum of affine functions is concave.)

SOLUTION

For any fixed $\mathbf{x}$, the map $\boldsymbol{\alpha} \mapsto \mathcal{L}(\mathbf{x}, \boldsymbol{\alpha}) = f(\mathbf{x}) + \sum_i \alpha_i g_i(\mathbf{x})$ is *affine* in $\boldsymbol{\alpha}$ (linear with constant offset $f(\mathbf{x})$), hence both convex and concave in $\boldsymbol{\alpha}$.

The dual function is $d(\boldsymbol{\alpha}) = \inf_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\alpha})$, the pointwise infimum of a family of affine (hence concave) functions. The pointwise infimum of any family of concave functions is concave: for any $\boldsymbol{\alpha}_1, \boldsymbol{\alpha}_2$ and $t \in [0, 1]$,

$$\begin{aligned} d(t\boldsymbol{\alpha}_1 + (1-t)\boldsymbol{\alpha}_2) &= \inf_{\mathbf{x}} \mathcal{L}(\mathbf{x}, t\boldsymbol{\alpha}_1 + (1-t)\boldsymbol{\alpha}_2) \\ &= \inf_{\mathbf{x}} [t\mathcal{L}(\mathbf{x}, \boldsymbol{\alpha}_1) + (1-t)\mathcal{L}(\mathbf{x}, \boldsymbol{\alpha}_2)] \\ &\geq t \inf_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\alpha}_1) + (1-t) \inf_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\alpha}_2) \\ &= td(\boldsymbol{\alpha}_1) + (1-t)d(\boldsymbol{\alpha}_2). \quad \square \end{aligned}$$

Exercise 5.26 (Open-ended). Compare Adam and SGD with momentum on a simple non-convex function such as the Rosenbrock banana function $\mathcal{L}(w_1, w_2) = (1 - w_1)^2 + 100(w_2 - w_1^2)^2$. Which converges faster? What do the trajectories look like?

Further Reading

The heavy-ball method originates with Polyak [1964]. Nesterov's acceleration and the estimate sequence proof are in Nesterov [2018]. Adam was introduced in Kingma and Ba [2015]; its non-convergence and AMSGrad fix are in Reddi et al. [2018]. Convergence of SGD is surveyed in Bottou et al. [2018]. The PL condition and its relation to overparameterized neural networks is studied in Liu et al. [2022]. For a unified treatment of first-order methods with optimal rates, see Bubeck [2015].